ĐẠI HỌC PHENIKAA
TRƯỜNG CÔNG NGHỆ THÔNG TIN PHENIKAA



BÁO CÁO BÀI TẬP LỚN

LẬP TRÌNH DI ĐỘNG

TÊN DỰ ÁN: XÂY DỰNG ỨNG DỤNG QUẢN LÝ TÀI CHÍNH CÁ NHÂN

(Expense Tracker — Flutter & Firebase)

Mã học phần: CSE702027 · Lớp: CSE702027-1-1-26(N05)

Họ và tên	*Mã số sinh viên*	*E-mail*
Phạm Hải Đăng	***24100454***	***24100454@st.phenikaa-uni.edu.vn***

GVHD: ……

Hà Nội, tháng 11 năm 2026

THÔNG TIN BÀI NỘP

LẬP TRÌNH DI ĐỘNG · CSE702027-1-1-26(N05)

Nội dung	Thông tin
Học phần	LẬP TRÌNH DI ĐỘNG (CSE702027)
Lớp	CSE702027-1-1-26(N05)
Đề tài	XÂY DỰNG ỨNG DỤNG QUẢN LÝ TÀI CHÍNH CÁ NHÂN
Video demo	…… (dán link trước khi nộp)
Kho mã nguồn	https://github.com/TBDD-G3/Project-Mobile
Nhánh nộp bài	develop
Ngày hoàn thành	tháng 11 năm 2026

Bảng đóng góp của nhóm

Thành viên	MSSV	Phần việc đảm nhận
Dương Công Minh	22010009	Xác thực, Quản lý ví, Tầng ORM Firestore, Màn hình Phiên Đấu Giá, kiểm thử và báo cáo
Phạm Hải Đăng	24100454	Giao dịch thu/chi, danh mục, lọc và tìm kiếm giao dịch
Đinh Đức Thịnh	24100005	Ngân sách, cảnh báo vượt hạn mức, biểu đồ báo cáo

Ghi chú: báo cáo được in thành 03 bản, mỗi bản một bìa mang tên và mã số của một thành viên; nội dung bên trong giống nhau. Bản PDF nộp trên Canvas là bản của nhóm trưởng.

MỤC LỤC

Chương 1. Giới thiệu 6
1.1. Đặt vấn đề 6
1.2. Mục tiêu và phạm vi 7
1.3. Công nghệ sử dụng 8
1.4. Bố cục báo cáo 8
Chương 2. Phân tích yêu cầu 9
2.1. Tác nhân của hệ thống 9
2.2. User stories (Câu 1) 9
2.3. Phân tích đối tượng (Câu 2) 10
2.3.1. AppUser — người dùng 11
2.3.2. Wallet — ví tiền 11
2.3.3. Category — danh mục 12
2.3.4. Transaction — giao dịch 12
2.3.5. Budget — ngân sách 13
2.4. Quan hệ giữa các đối tượng 14
2.5. Yêu cầu phi chức năng 14
Chương 3. Thiết kế hệ thống 16
3.1. Kiến trúc tổng thể 16
3.2. Sơ đồ lớp (Câu 3) 16
3.3. Sơ đồ tuần tự 17
3.3.1. Đăng ký và đăng nhập 18
3.3.2. Thêm giao dịch và cập nhật số dư 19
3.3.3. Đặt ngân sách và cảnh báo vượt hạn mức 20
3.3.4. Xem báo cáo và biểu đồ 21
3.3.5. Sửa và xoá giao dịch 21
3.4. Wireframe các màn hình (Câu 4) 22
3.5. Luồng làm việc của từng màn hình 24

Chương 4. Cài đặt 26
4.1. Môi trường và cấu trúc mã nguồn 26
4.2. Màn hình Phiên Đấu Giá (Câu 5) 27
4.3. Triển khai 15 user story (Câu 6) 28
4.3.1. Xác thực và tài khoản (US01 – US04) 29
4.3.2. Quản lý ví (US05 – US06) 30
4.3.3. Giao dịch thu chi (US07 – US11) 32
4.3.4. Danh mục (US12) 33
4.3.5. Ngân sách và cảnh báo (US13 – US14) 34
4.3.6. Báo cáo và biểu đồ (US15) 35
4.4. Firebase và tầng ORM Firestore (Câu 7) 36
4.4.1. Cấu trúc dữ liệu 36
4.4.2. Ánh xạ document sang model bằng withConverter 37
4.4.3. Repository tách khỏi giao diện 37
4.4.4. Cập nhật số dư bằng FieldValue.increment 38
4.4.5. Security Rules 38
4.5. Kiểm tra dữ liệu và xử lý lỗi (Câu 8a) 39
Chương 5. Kiểm thử 41
5.1. Chiến lược kiểm thử 41
5.2. Unit test (8b) 41
5.3. Widget test (8c) 42
5.4. Phân tích ca lỗi và ca thành công (8d) 43
5.5. Bảng test case thủ công (8a) 44
5.6. Kiểm thử giao diện màn hình (8e) 44
5.7. Tổng hợp kết quả 46
Chương 6. Đạo đức và quyền riêng tư dữ liệu 47
6.1. Vì sao ứng dụng tài chính phải đặt riêng tư lên trước 47
6.2. Sự việc 22/10/2026: Facebook bị kiện 25 tỷ USD về dữ liệu cá nhân 47

6.3. Khung pháp lý 48
6.3.1. GDPR của Liên minh châu Âu 48
6.3.2. Pháp luật Việt Nam 48
6.4. Áp dụng vào ứng dụng quản lý tài chính cá nhân 48
6.4.1. Tối thiểu dữ liệu 48
6.4.2. Kiểm soát truy cập bằng Security Rules 49
6.4.3. Quyền xoá dữ liệu — khoảng trống đã biết 49
6.4.4. Những chỗ cố ý không làm 49
6.5. Liên hệ với nội dung thảo luận trên lớp 49
Chương 7. Kết luận và hướng phát triển 51
7.1. Kết quả đạt được 51
7.2. Hạn chế 51
7.3. Hướng phát triển 52
Chương 8. Phân công công việc và đóng góp (Câu 10) 53
Tài liệu tham khảo 55
Phụ lục A. Hướng dẫn chạy chương trình 56
Phụ lục B. Khai báo sử dụng công cụ hỗ trợ 57

DANH MỤC HÌNH ẢNH

Hình 1. Kiến trúc ba lớp của ứng dụng....16
Hình 2. Sơ đồ lớp của ứng dụng....17
Hình 3. Sơ đồ tuần tự luồng đăng nhập....18
Hình 4. Sơ đồ tuần tự luồng thêm giao dịch....19
Hình 5. Sơ đồ tuần tự luồng đặt ngân sách và cảnh báo....20
Hình 6. Sơ đồ tuần tự luồng xem báo cáo....21
Hình 7. Sơ đồ tuần tự luồng sửa và xoá giao dịch....21
Hình 8. Wireframe màn đăng nhập và đăng ký....22
Hình 9. Wireframe màn tổng quan danh sách ví....22
Hình 10. Wireframe màn thêm giao dịch....22
Hình 11. Wireframe màn danh sách giao dịch....23
Hình 12. Wireframe màn ngân sách có cảnh báo vượt....23
Hình 13. Wireframe màn báo cáo và biểu đồ....23
Hình 14. Wireframe màn hồ sơ và đăng xuất....23
Hình 15. Sơ đồ điều hướng giữa các màn hình....24
Hình 16. Màn hình Phiên Đấu Giá chạy trên ứng dụng (Câu 5)....28
Hình 17. Màn đăng nhập chạy trên Chrome....30
Hình 18. Màn quản lý ví chạy trên ứng dụng....31
Hình 19. Tab Giao dịch với dữ liệu mẫu: bộ lọc, tổng tháng và danh sách theo ngày.32
Hình 20. Form thêm giao dịch trong bottom sheet....33
Hình 21. Màn Danh mục: danh mục mặc định có ổ khoá, FAB thêm danh mục....34
Hình 22. Tab Ngân sách với dữ liệu mẫu: ba mức xanh – vàng – đỏ....35
Hình 23. Tab Báo cáo với dữ liệu mẫu: biểu đồ tròn và cột....36
Hình 24. Màn đăng nhập....45
Hình 25. Tab Ví tiền....45
Hình 26. Tab Giao dịch....45
Hình 27. Màn Danh mục....45
Hình 28. Tab Ngân sách....45
Hình 29. Tab Báo cáo....45
Hình 30. Tab Đấu giá....46

DANH MỤC BẢNG BIỂU

Bảng 1. *Công nghệ và vai trò trong dự án*.....8
Bảng 2. *15 user story của ứng dụng (trạng thái tại thời điểm viết báo cáo)*.....9
Bảng 3. *Thuộc tính của lớp AppUser*.....11
Bảng 4. *Thuộc tính của lớp Wallet*.....11
Bảng 5. *Thuộc tính của lớp Category*.....12
Bảng 6. *Thuộc tính của lớp Transaction*.....13
Bảng 7. *Thuộc tính của lớp Budget*.....13
Bảng 8. *Quan hệ giữa năm đối tượng*.....14
Bảng 9. *Yêu cầu phi chức năng*.....14
Bảng 10. *Luồng làm việc của từng màn hình*.....24
Bảng 11. *Môi trường phát triển*.....26
Bảng 12. *Ánh xạ user story sang màn hình và tệp mã nguồn*.....28
Bảng 13. *Năm vị trí lưu dữ liệu trên Firestore*.....36
Bảng 14. *Các tình huống lỗi và thông báo tương ứng*.....39
Bảng 15. *Bốn mức kiểm thử*.....41
Bảng 16. *Các nhóm unit test hiện có*.....41
Bảng 17. *Kịch bản widget test*.....42
Bảng 18. *Ca thành công và ca lỗi đã quan sát*.....43
Bảng 19. *Bảng test case thủ công*.....44
Bảng 20. *Kế hoạch và phân công theo tuần*.....53
Bảng 21. *Đóng góp của từng thành viên*.....53
Bảng 22. *Thống kê commit thật (70 commit, mọi nhánh)*.....53
Bảng 23. *Tám commit gần nhất trên develop*.....54
Bảng 24. *Công cụ hỗ trợ đã dùng*.....57

Chương 1. Giới thiệu

1.1. Đặt vấn đề

Quản lý chi tiêu cá nhân là việc ai cũng phải làm nhưng ít người làm đều. Phần lớn sinh viên và người đi làm ghi chép tạm bằng ứng dụng ghi chú, ảnh chụp hoá đơn hoặc sổ tay, nên đến cuối tháng không biết tiền đã đi đâu. Ba câu hỏi thường gặp là: tháng này mình tiêu hết bao nhiêu, tiêu nhiều nhất vào việc gì, và tháng sau nên cắt

khoản nào. Một cuốn sổ giấy trả lời được câu thứ nhất nhưng không trả lời được hai câu còn lại, vì muốn biết thì phải cộng lại toàn bộ.

Các ứng dụng quản lý tài chính trên thị trường chia làm hai nhóm. Nhóm thứ nhất gắn với ngân hàng, tự động lấy giao dịch nhưng yêu cầu liên kết tài khoản và thường chỉ dùng được với một số ngân hàng. Nhóm thứ hai cho nhập tay hoàn toàn, chủ động và riêng tư hơn, nhưng nhiều ứng dụng bắt trả phí để xem báo cáo hoặc chặn quảng cáo. Với phạm vi một bài tập lớn, nhóm chọn hướng thứ hai: người dùng tự ghi giao dịch, ứng dụng lo phần tổng hợp, cảnh báo và trực quan hoá.

Đề tài đặt ra là xây dựng ứng dụng di động **Expense Tracker** trên nền Flutter, lưu dữ liệu trên Firebase, cho phép quản lý nhiều ví, ghi giao dịch thu chi theo danh mục, đặt hạn mức chi tiêu theo tháng và xem báo cáo trực quan. Ngoài các chức năng nghiệp vụ, ứng dụng còn có màn hình Phiên Đấu Giá theo mẫu cho sẵn để luyện kỹ năng dựng giao diện và xử lý bộ đếm thời gian.

1.2. Mục tiêu và phạm vi

Mục tiêu của bài tập lớn gồm bốn điểm:

- Phân tích được yêu cầu thành user story, xác định đối tượng dữ liệu và quan hệ giữa chúng.
- Thiết kế sơ đồ lớp, sơ đồ tuần tự cho các luồng chính và wireframe cho các màn hình.
- Cài đặt ứng dụng chạy được trên thực tế: đăng ký, đăng nhập, quản lý ví, ghi giao dịch, đặt ngân sách, xem báo cáo, đăng xuất.
- Kiểm thử bằng unit test, widget test và bảng test case thủ công, có xử lý lỗi rõ ràng.

Trong phạm vi: 15 user story được trình bày ở mục 2.2, màn hình Phiên Đấu Giá, và các phần kiểm thử, báo cáo, video demo theo yêu cầu học phần.

Ngoài phạm vi: ứng dụng không liên kết trực tiếp với ngân hàng, không đồng bộ nhiều thiết bị theo thời gian thực ngoài cơ chế có sẵn của Firestore, không hỗ trợ nhiều loại tiền tệ cùng lúc (mặc định VND), không có tính năng chia sẻ ví cho người khác và không xử lý thanh toán thật trong phiên đấu giá.

Báo cáo viết trên trạng thái mã nguồn tại nhánh `develop` ngày 01/11/2026. Toàn bộ 15 user story đã gộp và chạy trên Firestore; nhánh `main` được đồng bộ theo `develop` sau khi gộp.

1.3. Công nghệ sử dụng

Bảng 1. *Công nghệ và vai trò trong dự án*

Thành phần	Công nghệ	Vai trò trong ứng dụng
Giao diện	Flutter 3.47, Dart 3.13	Dựng toàn bộ màn hình; chạy trên Android, iOS và web. Firebase không hỗ trợ Windows/Linux desktop nên hai nền tảng này không dùng để chạy thử
Quản lý trạng thái	flutter_riverpod 2.6	Tách trạng thái khỏi giao diện, tự động dựng lại khi dữ liệu đổi
Xác thực	Firebase Authentication	Đăng ký, đăng nhập, đặt lại mật khẩu bằng email
Cơ sở dữ liệu	Cloud Firestore 7.0	5 collection phẳng, đọc bằng stream, ghi từng Document qua lớp ORM
Biểu đồ	fl_chart 0.70	Vẽ biểu đồ tròn và biểu đồ cột ở màn Báo cáo
Định dạng	intl 0.20	Định dạng tiền tệ VND và ngày theo tiếng Việt

1.4. Bố cục báo cáo

Báo cáo gồm tám chương. Chương 1 giới thiệu bài toán và công nghệ. Chương 2 trình bày kết quả phân tích yêu cầu: tác nhân, 15 user story, 5 đối tượng dữ liệu và quan hệ giữa chúng. Chương 3 trình bày thiết kế: kiến trúc ba lớp, sơ đồ lớp, năm sơ đồ tuần tự, wireframe bảy màn hình và sơ đồ điều hướng. Chương 4 trình bày phần cài đặt, gắn từng user story với màn hình và tệp mã nguồn cụ thể, kèm phần mô tả tầng ORM Firestore và Security Rules. Chương 5 trình bày chiến lược và kết quả kiểm thử. Chương 6 là bài luận về đạo đức và quyền riêng tư dữ liệu. Chương 7 kết luận và nêu hướng phát triển. Chương 8 là bảng phân công công việc của ba thành viên.

Chương 2. Phân tích yêu cầu

2.1. Tác nhân của hệ thống

Ứng dụng có hai tác nhân chính là người dùng và hệ thống. Người dùng chưa đăng nhập (khách) chỉ tiếp cận được ba màn hình xác thực: đăng nhập, đăng ký, quên mật khẩu. Người dùng đã đăng nhập mới thấy dữ liệu tài chính của chính mình. Ngoài ra còn một tác nhân phụ là **Firebase**: hệ thống không tự sinh dữ liệu mà phản ứng theo thay đổi từ Firestore, nên khi mô tả luồng cần chỉ rõ điểm nào dữ liệu được ghi và điểm nào được đọc lại.

- **Khách:** xem màn hình đăng nhập, đăng ký tài khoản mới, yêu cầu gửi email đặt lại mật khẩu.
- **Người dùng:** quản lý ví, ghi giao dịch, quản lý danh mục, đặt ngân sách, xem báo cáo, đăng xuất.
- **Firebase:** xác thực danh tính và lưu dữ liệu; là nguồn phát stream để giao diện cập nhật.

2.2. User stories (Câu 1)

Nhóm ba thành viên nên yêu cầu tối thiểu là 12 user story; nhóm thực hiện **15 user story** để chia đều mỗi người năm câu và có dư địa khi một chức năng phải cắt bớt. Mỗi câu viết theo mẫu "Là <vai trò>, tôi có thể <hành động> để <giá trị>". Cột Ưu tiên xếp theo thứ tự thực hiện: P1 là chức năng bắt buộc, P2 là chức năng tạo khác biệt, P3 là phần mở rộng. Cột Trạng thái cho biết câu nào đã chạy trên `develop`, câu nào còn ở nhánh của thành viên phụ trách.

Bảng 2. *15 user story của ứng dụng (trạng thái tại thời điểm viết báo cáo)*

Mã	Vai trò	Hành động	Giá trị	Ưu tiên	Trạng thái
US01	Khách	Đăng ký tài khoản bằng email và mật khẩu	Dùng được ứng dụng ngay mà không cần chờ duyệt	P1	Đã chạy
US02	Người dùng	Đăng nhập bằng email và mật khẩu	Vào được dữ liệu tài chính của mình	P1	Đã chạy
US03	Người dùng	Yêu cầu gửi email đặt lại mật khẩu	Lấy lại được tài khoản khi quên mật khẩu	P1	Đã chạy
US04	Người dùng	Đăng xuất khỏi ứng dụng	Bảo vệ dữ liệu khi dùng máy chung	P1	Đã chạy
US05	Người dùng	Tạo ví tiền với tên, loại và số dư ban đầu	Tách được tiền mặt, ngân hàng và ví điện tử	P1	Đã chạy

Mã	Vai trò	Hành động	Giá trị	Ưu tiên	Trạng thái
US06	Người dùng	Sửa tên hoặc xoá ví không còn dùng	Danh sách ví giữ đúng thực tế	P1	Đã chạy
US07	Người dùng	Thêm giao dịch thu hoặc chi kèm danh mục và ghi chú	Ghi lại mọi khoản phát sinh trong ngày	P1	Đã chạy
US08	Người dùng	Sửa một giao dịch đã ghi	Chữa được lỗi nhập sai số tiền	P1	Đã chạy
US09	Người dùng	Xoá một giao dịch đã ghi	Bỏ được giao dịch trùng hoặc ghi nhầm	P1	Đã chạy
US10	Người dùng	Xem danh sách giao dịch theo từng tháng	Đối chiếu được chi tiêu của tháng	P1	Đã chạy
US11	Người dùng	Lọc giao dịch theo danh mục và loại	Tìm nhanh khoản cần kiểm tra	P2	Đã chạy
US12	Người dùng	Tạo danh mục chi tiêu riêng	Phân loại theo cách chi tiêu của mình	P2	Đã chạy
US13	Người dùng	Đặt hạn mức chi tiêu tháng cho một danh mục	Kiểm soát được khoản dễ vượt nhất	P2	Đã chạy
US14	Người dùng	Nhận cảnh báo khi chi gần hoặc vượt hạn mức	Điều chỉnh chi tiêu trước khi hết tháng	P2	Đã chạy
US15	Người dùng	Xem biểu đồ thu chi theo danh mục và theo tháng	Thấy được thói quen chi tiêu của mình	P3	Đã chạy

Mười câu ưu tiên P1 tạo thành luồng sử dụng tối thiểu: đăng ký, đăng nhập, tạo ví, ghi giao dịch, sửa hoặc xoá, xem theo tháng và đăng xuất. Nếu chỉ hoàn thành nhóm này, ứng dụng đã thay thế được sổ ghi chép. Bốn câu P2 trả lời hai câu hỏi còn lại của người dùng (tiêu nhiều nhất vào đâu, tháng sau cắt khoản nào), còn US15 là phần trực quan hoá nên xếp sau cùng để không chặn các câu khác.

Tại thời điểm viết báo cáo, cả 15 user story chạy được trên `develop` với Firestore thật. Màn hình Phiên Đấu Giá của Câu 5 đã chạy nhưng không nằm trong danh sách user story vì đây là màn hình mẫu, dữ liệu nằm trong bộ nhớ.

2.3. Phân tích đối tượng (Câu 2)

Dữ liệu của ứng dụng gồm **5 đối tượng chính**. Mỗi đối tượng là một lớp Dart trong thư mục `lib/data/firestore/models` và tương ứng một collection phẳng trên Firestore. Cả năm lớp đều có `fromDocument` (đọc document thành đối tượng) và `toDocument` (chuyển đối tượng thành map để ghi), bốn lớp có thêm `copyWith` để tạo bản sao khi chỉ đổi vài trường.

2.3.1. AppUser — người dùng

Lưu hồ sơ người dùng trong collection `users`, mã document trùng với uid của Firebase Authentication. Mật khẩu không lưu ở đây vì Firebase Authentication đã quản lý; đây là điểm quan trọng về bảo mật, được phân tích thêm ở Chương 6. Lớp chỉ có `fromDocument` và `toDocument`, không có `copyWith` vì hồ sơ ít khi sửa từng phần.

Bảng 3. *Thuộc tính của lớp AppUser*

Thuộc tính	Kiểu	Mô tả
id	String	Trùng uid của Firebase Authentication
email	String	Email dùng để đăng nhập
displayName	String	Tên hiển thị, lấy từ lúc đăng ký
currencyCode	String	Mã tiền tệ, mặc định VND
createdAt	DateTime	Thời điểm tạo tài khoản

Phương thức hoạt động: `AuthService.register()` gọi `createUserWithEmailAndPassword` rồi ghi hồ sơ vào `users/{uid}`; `signIn()` đăng nhập; `sendPasswordReset()` gửi email đặt lại mật khẩu; `signOut()` đăng xuất; `authStateChanges` trả stream trạng thái đăng nhập và đọc hồ sơ tương ứng. `AuthGate` nghe stream này để quyết định hiện `LoginScreen` hay `MainNavScreen`.

2.3.2. Wallet — ví tiền

Một ví thuộc đúng một người dùng và một trong ba loại: `cash` (tiền mặt), `bank` (tài khoản ngân hàng), `ewallet` (ví điện tử). Số dư **không sửa tay trên giao diện** mà chỉ thay đổi khi thêm, sửa hoặc xoá giao dịch; quy tắc này giữ cho số dư luôn khớp với lịch sử giao dịch.

Bảng 4. *Thuộc tính của lớp Wallet*

Thuộc tính	Kiểu	Mô tả
id	String	Mã ví, sinh bằng uuid khi tạo
userId	String	Chủ sở hữu ví
name	String	Tên ví, ví dụ "Tiền mặt", "Vietcombank"
type	String	cash | bank | ewallet
balance	double	Số dư hiện tại, đơn vị đồng
iconCode	int?	Mã icon hiển thị trong danh sách
colorHex	String?	Màu nhận diện của ví, dạng #RRGGBB
createdAt	DateTime	Dùng để sắp xếp danh sách ví

Phương thức hoạt động: interface `WalletRepository` khai báo `watchAllWallets()`, `getWalletById()`, `insertWallet()`, `updateWallet()`, `updateBalance(walletId, delta)` và `deleteWallet()`. Lớp `FirestoreWalletRepository` lọc `where('userId', isEqualTo: uid)`, sắp xếp theo `createdAt`; riêng `updateBalance` dùng `FieldValue.increment(delta)` để cộng trừ nguyên tử trên server, tránh trường hợp hai lần ghi liên tiếp làm mất số dư. `MockWalletRepository` cài cùng interface bằng List + StreamController để test không cần Firebase.

2.3.3. Category — danh mục

Phân loại giao dịch thu hoặc chi. Ứng dụng có sẵn 13 danh mục mặc định (`isDefault = true`) do seeder tạo với id cố định, và người dùng tạo thêm danh mục riêng. Danh mục không gắn với ví vì cùng một khoản ăn uống có thể trả bằng tiền mặt hoặc ví điện tử. Lớp **không có `userId`**: collection `categories` dùng chung cho mọi tài khoản, rule chỉ yêu cầu đã đăng nhập.

Bảng 5. *Thuộc tính của lớp Category*

Thuộc tính	Kiểu	Mô tả
id	String	Mã danh mục; danh mục mặc định dùng id cố định
name	String	Tên danh mục, ví dụ "Ăn uống"
type	String	income | expense
iconCode	int?	Mã codepoint của Material Icon
colorHex	String?	Màu dùng trong biểu đồ tròn
isDefault	bool	Đánh dấu danh mục seed sẵn

Phương thức hoạt động: ngoài `fromDocument`/`toDocument`/`copyWith(name, iconCode, colorHex)`, lớp có hai getter `isExpense` và `isIncome` để giao diện chọn đúng danh sách danh mục theo loại giao dịch. Interface `CategoryRepository` gồm `watchAll()`, `getAll()`, `getById()`, `insertCategory()`, `updateCategory()` và `deleteCategory()`.

2.3.4. Transaction — giao dịch

Ghi lại một khoản thu, chi hoặc chuyển khoản gắn với một ví. Trường `amount` luôn dương; dấu cộng trừ do `type` quyết định, nhờ vậy khi hiển thị chỉ cần đổi dấu theo loại thay vì lưu số âm trong cơ sở dữ liệu.

Bảng 6. *Thuộc tính của lớp Transaction*

Thuộc tính	Kiểu	Mô tả
id	String	Mã giao dịch
userId	String	Người tạo giao dịch
walletId	String	Ví phát sinh giao dịch
categoryId	String?	Danh mục; có thể null nếu người dùng không chọn
type	String	income | expense | transfer
amount	double	Số tiền, luôn dương
date	DateTime	Ngày giao dịch, dùng để lọc theo tháng
note	String?	Ghi chú tự do
toWalletId	String?	Ví nhận tiền, chỉ dùng khi type là transfer

Phương thức hoạt động: ba getter `isExpense`, `isIncome`, `isTransfer` cho biết loại giao dịch; dấu tiền được suy ra từ đó (thu cộng, chi trừ). Interface `TransactionRepository` gồm `watchTransactionsByMonth()`, `watchByWallet()`, `insertTransaction()`, `updateTransaction()` và `deleteTransaction()`. Theo hợp đồng ghi trong interface, mỗi lần thêm, sửa hoặc xoá, repository phải gọi `walletRepository.updateBalance(walletId, delta)` để cập nhật số dư ví (trình bày ở mục 4.4).

2.3.5. Budget — ngân sách

Hạn mức chi tiêu cho một danh mục trong một tháng cụ thể. Một danh mục có thể có nhiều ngân sách ứng với các tháng khác nhau, nên khoá nghiệp vụ là bộ ba `categoryId`, `month`, `year`; `getBudgetByCategory` tra đúng bộ ba này để sửa thay vì tạo bản ghi trùng.

Bảng 7. *Thuộc tính của lớp Budget*

Thuộc tính	Kiểu	Mô tả
id	String	Mã ngân sách
userId	String	Người đặt ngân sách
categoryId	String	Danh mục được đặt hạn mức
month	int	Tháng, từ 1 đến 12
year	int	Năm
limitAmount	double	Hạn mức chi tiêu của tháng

Phương thức hoạt động: interface `BudgetRepository` gồm `watchBudgetsByMonth()`, `getBudgetByCategory()`, `insertBudget()`, `updateBudget()` và `deleteBudget()`. Phần tính tiến độ nằm trong lớp thuần `BudgetProgress` để unit test không cần Firestore; lớp này đã có trên `develop`.

2.4. Quan hệ giữa các đối tượng

Bảng 8. *Quan hệ giữa năm đối tượng*

Quan hệ	Bội số	Ý nghĩa
AppUser – Wallet	1 – n	Mỗi ví có userId chỉ chủ sở hữu; một người có nhiều ví
AppUser – Transaction	1 – n	Mỗi giao dịch có userId; danh sách tháng chỉ lấy của người đang đăng nhập
AppUser – Budget	1 – n	Mỗi ngân sách có userId, đặt cho nhiều danh mục và nhiều tháng
Wallet – Transaction	1 – n	Giao dịch gắn với một ví qua walletId, số dư ví đổi theo giao dịch
Category – Transaction	1 – n	Giao dịch thuộc một danh mục; categoryId có thể null
Category – Budget	1 – n	Ngân sách gắn với một danh mục, phân biệt theo month và year

Riêng giao dịch chuyển khoản tham chiếu tới hai ví: `walletId` là ví nguồn và `toWalletId` là ví nhận. Quan hệ này không vẽ thêm trong sơ đồ lớp để tránh rối, nhưng khi xử lý thì phải trừ số dư ví nguồn và cộng số dư ví nhận bằng hai lệnh `updateBalance` liên tiếp.

Về phân quyền dữ liệu: `wallets`, `transactions`, `budgets` đều có trường `userId` trong document và `firestore.rules` yêu cầu trường này trùng uid đang đăng nhập mới được đọc/ghi. Riêng `categories` là collection dùng chung nên rule chỉ yêu cầu `request.auth != null`.

2.5. Yêu cầu phi chức năng

Bảng 9. *Yêu cầu phi chức năng*

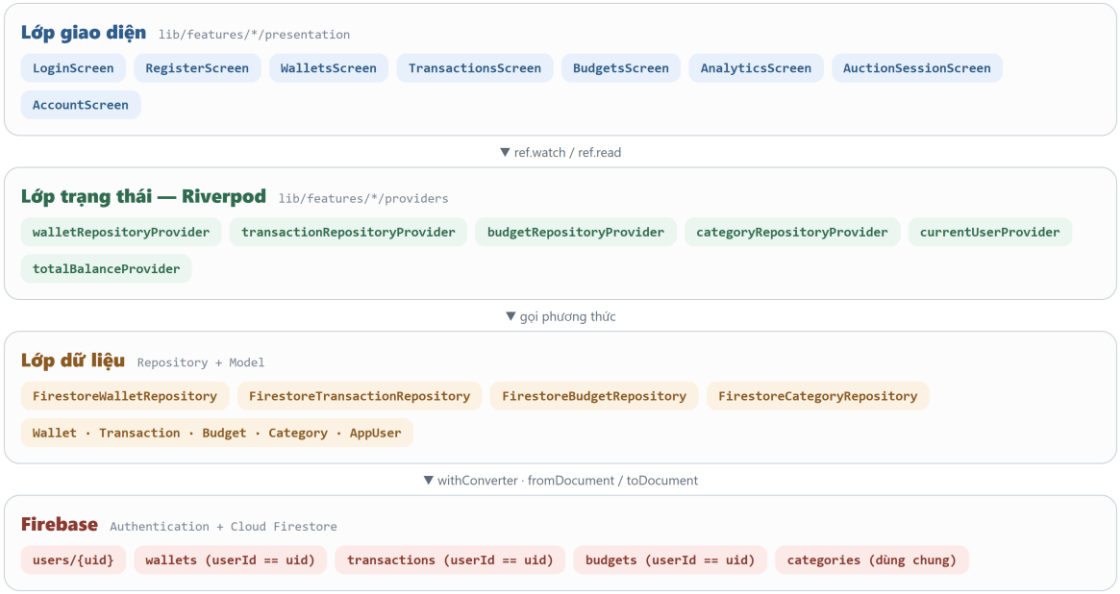
Nhóm	Yêu cầu
Hiệu năng	Danh sách giao dịch của một tháng hiển thị dưới 2 giây với 500 bản ghi; cuộn mượt trên thiết bị tầm trung
Bảo mật	Mỗi người dùng chỉ đọc và ghi được dữ liệu của mình; mật khẩu do Firebase quản lý; rules chặn truy cập chéo tài khoản
Toàn vẹn dữ liệu	Số dư ví cập nhật bằng phép cộng nguyên tử trên server, không ghi đè giá trị đọc về, nên hai thao tác liên tiếp không làm mất tiền

Nhóm	Yêu cầu
Khả dụng	Thông báo lỗi bằng tiếng Việt, chỉ rõ trường nào sai; thao tác xoá phải xác nhận
Tương thích	Chạy trên Android, iOS và web; màn hình từ 360 dp trở lên
Bảo trì	Tách ba lớp UI – Provider – Repository; mỗi lớp dữ liệu có một interface và một bản cài đặt Firestore

Chương 3. Thiết kế hệ thống

3.1. Kiến trúc tổng thể

Ứng dụng theo kiến trúc ba lớp, mỗi lớp chỉ biết lớp ngay dưới nó. Lớp giao diện không gọi thẳng Firebase mà đọc dữ liệu qua provider; provider gọi repository; repository mới là nơi duy nhất biết đường dẫn collection và câu lệnh ghi. Cách chia này có ba lợi ích: thay đổi giao diện không ảnh hưởng tầng dữ liệu, thay Firestore bằng nguồn khác chỉ cần viết lại repository, và test được từng lớp bằng đối tượng giả.



Hình 1. Kiến trúc ba lớp của ứng dụng

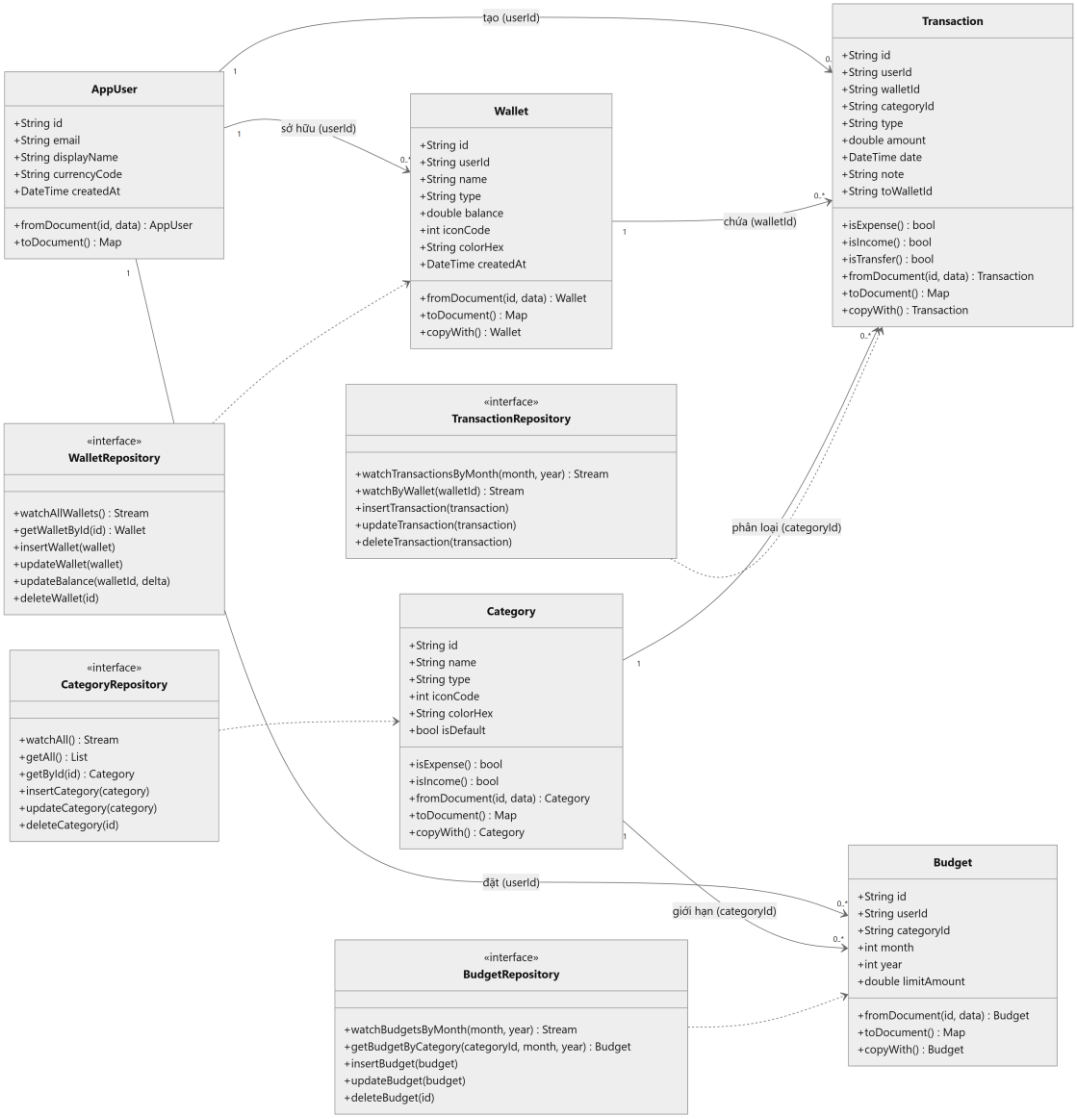
Toàn bộ trạng thái đi qua Riverpod. Bốn provider repository giữ một thực thể repository duy nhất cho cả ứng dụng, còn các provider dữ liệu như `walletsStreamProvider` hay `transactionsByMonthProvider` biến stream của Firestore thành `AsyncValue` để giao diện xử lý ba trạng thái: đang tải, có dữ liệu, lỗi. Nhờ vậy mỗi màn hình chỉ cần một khối `switch` trên `AsyncValue` thay vì tự quản lý cờ loading.

Mỗi model tự chuyển đổi với Document qua `fromDocument`/`toDocument`, còn `firestore_collections.dart` gắn sẵn `withConverter` để repository đọc ra thẳng object. Đây là phần trả lời Câu 7 và được phân tích kỹ ở mục 4.4.

3.2. Sơ đồ lớp (Câu 3)

Sơ đồ lớp gồm năm lớp dữ liệu và bốn interface repository. Tên lớp, thuộc tính và phương thức lấy đúng theo mã nguồn trong `lib/data/firestore/models` và

`lib/features/*/data/repositories`, không dùng tên giả định. Đường nét liền là quan hệ kết hợp có bội số, đường nét đứt là quan hệ phụ thuộc (repository làm việc với lớp dữ liệu tương ứng).



Hình 2. Sơ đồ lớp của ứng dụng

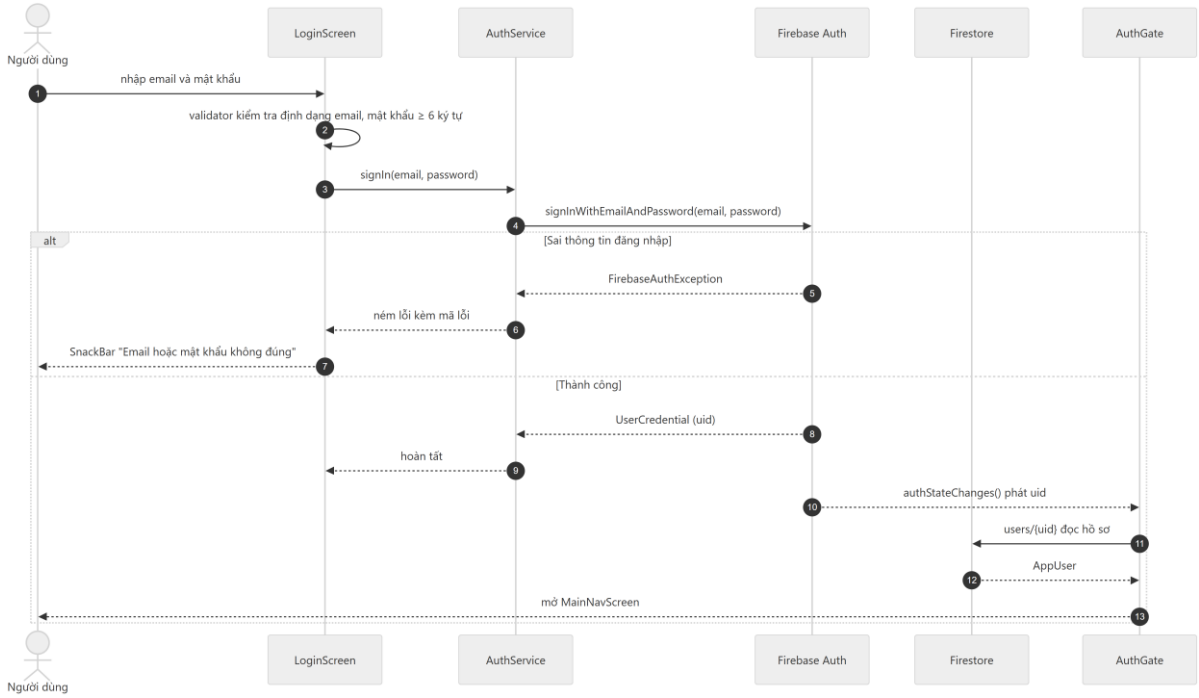
Điểm đáng chú ý trong sơ đồ là chiều phụ thuộc một chiều từ repository xuống model: model không biết repository nào đang dùng nó, nên có thể kiểm thử model độc lập. Bốn interface repository được khai báo trong thư mục feature tương ứng chứ không gom vào một chỗ, vì mỗi thành viên phụ trách một nhóm chức năng cần đọc được hợp đồng của mình mà không phải mở thư mục của người khác.

3.3. Sơ đồ tuần tự

Năm luồng dưới đây là những luồng có xử lý dữ liệu phức tạp nhất, cũng là những luồng dễ sai nếu chỉ đọc giao diện. Mỗi sơ đồ ghi rõ thời điểm ghi Firestore và

thời điểm giao diện được cập nhật lại. Hai luồng ngân sách và báo cáo là phần Member 3, đã có mã nguồn trên `develop`.

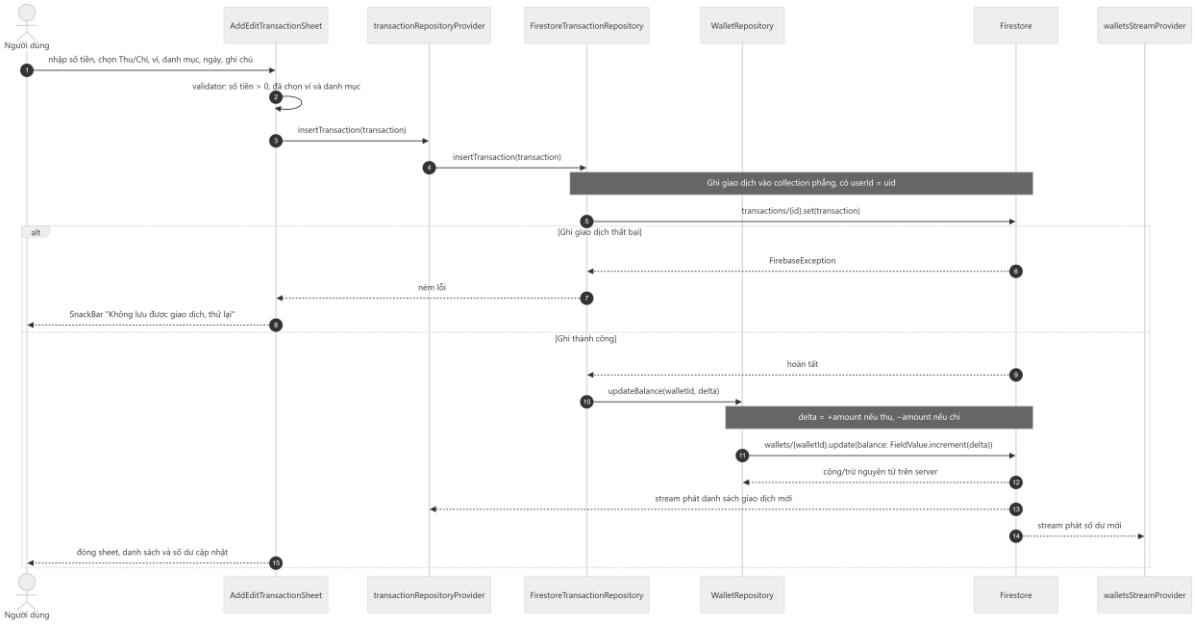
3.3.1. Đăng ký và đăng nhập



Hình 3. Sơ đồ tuần tự luồng đăng nhập

Điểm cần lưu ý là AuthGate chỉ là một widget lắng nghe `authStateChanges`, không tự quyết định điều hướng. Khi đăng nhập thành công, Firebase phát sự kiện, AuthGate đọc hồ sơ `users/{uid}` rồi mới dựng MainNavScreen. Nhờ vậy nếu người dùng đăng xuất ở bất kỳ màn hình nào, giao diện tự quay về màn đăng nhập mà không cần gọi `Navigator` thủ công.

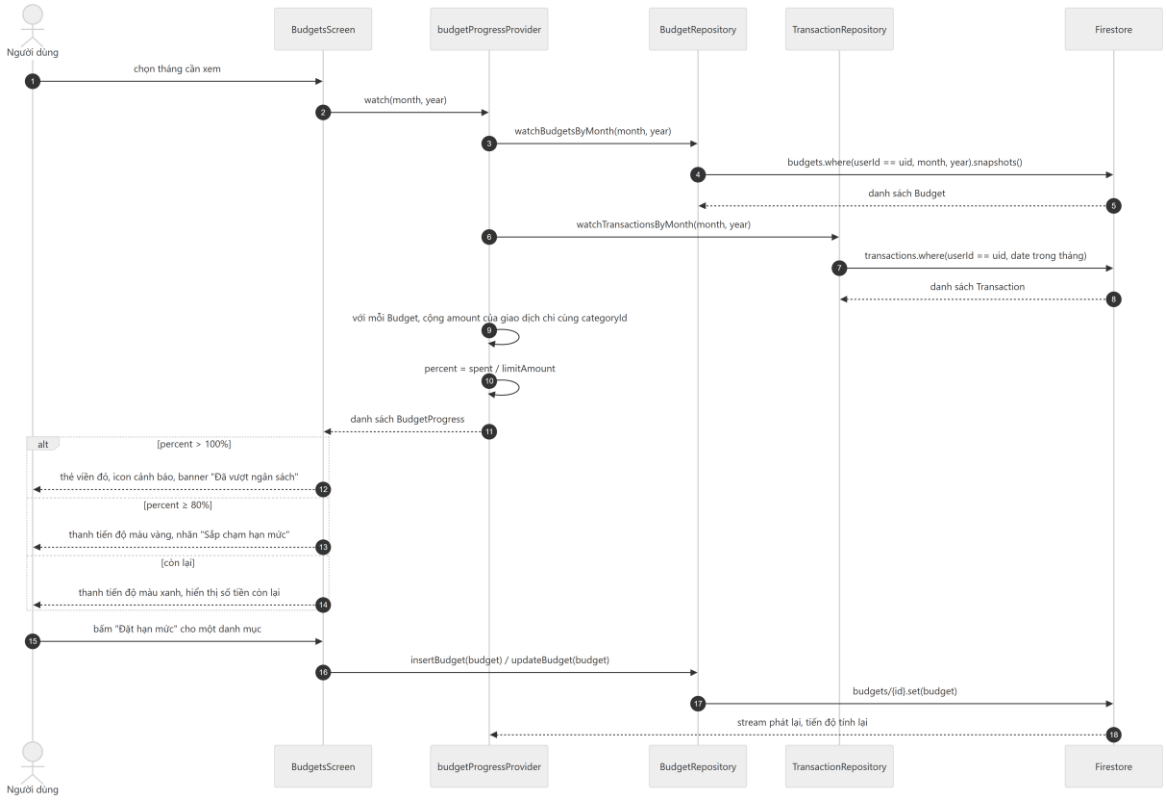
3.3.2. Thêm giao dịch và cập nhật số dư



Hình 4. Sơ đồ tuần tự luồng thêm giao dịch

Đây là luồng quan trọng nhất của ứng dụng. Giao dịch được ghi trước, sau đó repository gọi `updateBalance(walletId, delta)` để cộng hoặc trừ số dư. Lệnh thứ hai dùng `FieldValue.increment(delta)` nên phép cộng diễn ra trên server, không phụ thuộc giá trị số dư mà máy khách đọc được — hai lần ghi liên tiếp vẫn cho kết quả đúng. Thiết kế chấp nhận rủi ro nhỏ: nếu mất mạng đúng giữa hai lệnh thì giao dịch đã ghi mà số dư chưa đổi; khi đó lần đọc lại danh sách ví sẽ thấy chênh và có thể sửa bằng cách cập nhật lại số dư theo lịch sử giao dịch. Hướng khắc phục triệt để là gộp hai lệnh vào một `WriteBatch`, được nêu ở mục 7.3.

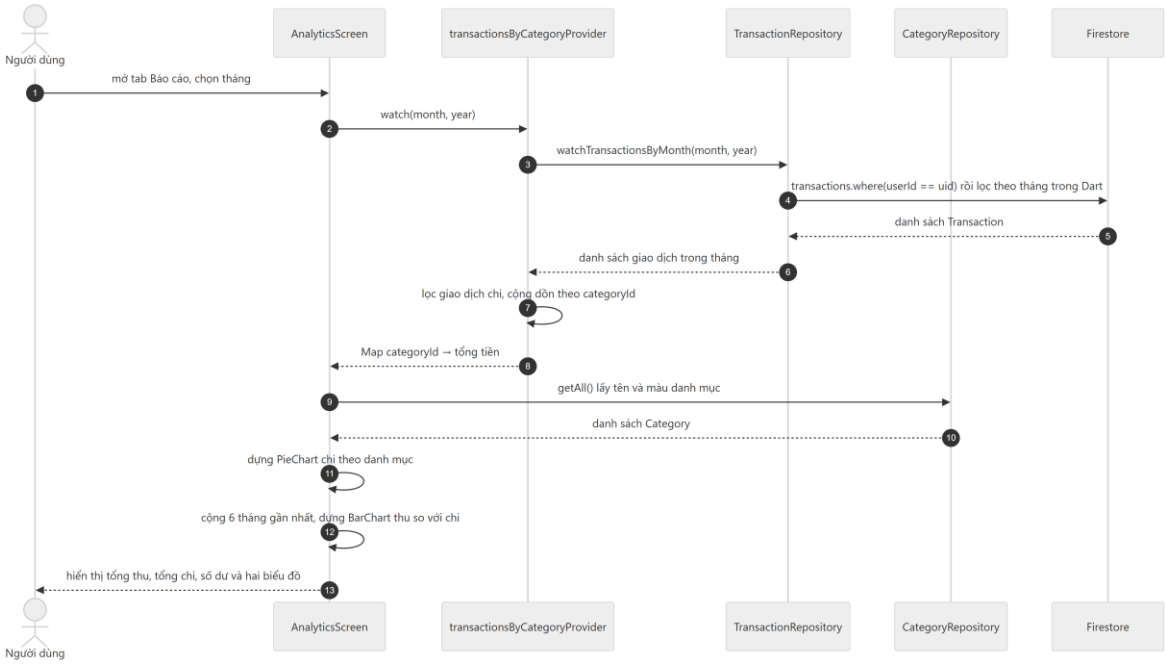
3.3.3. Đặt ngân sách và cảnh báo vượt hạn mức



Hình 5. Sơ đồ tuần tự luồng đặt ngân sách và cảnh báo

Ngân sách không lưu số đã chi mà tính lại mỗi khi dữ liệu đổi. Cách này tránh được lỗi cộng dồn hai lần khi sửa giao dịch, đổi lại tốn một lần duyệt danh sách giao dịch trong tháng — với vài trăm bản ghi thì chi phí này không đáng kể. Ngưỡng cảnh báo chia ba mức: dưới 80% hiển thị xanh, từ 80% đến 100% hiển thị vàng kèm nhãn "sắp chạm hạn mức", trên 100% hiển thị đỏ kèm banner.

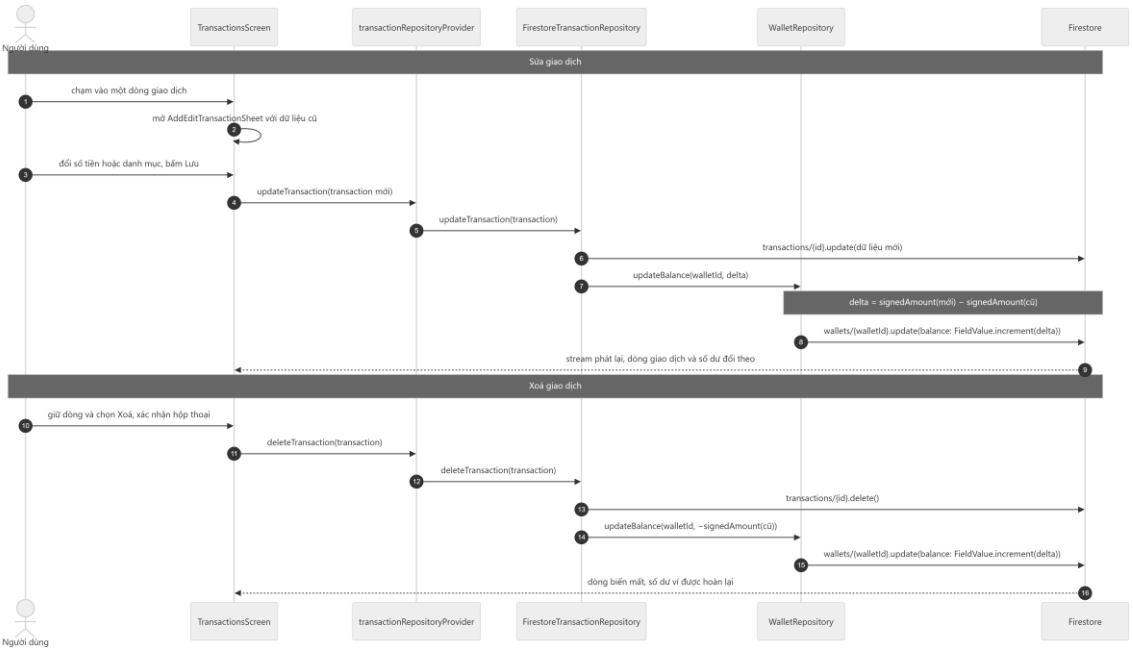
3.3.4. Xem báo cáo và biểu đồ



Hình 6. Sơ đồ tuần tự luồng xem báo cáo

Màn báo cáo dùng lại provider tính tổng theo danh mục của lớp giao dịch, sau đó ghép với tên và màu danh mục để vẽ biểu đồ tròn. Biểu đồ cột lấy dữ liệu sáu tháng gần nhất bằng sáu lần đọc theo tháng, không đọc toàn bộ collection, để chi phí đọc Firestore không tăng theo thời gian sử dụng.

3.3.5. Sửa và xoá giao dịch

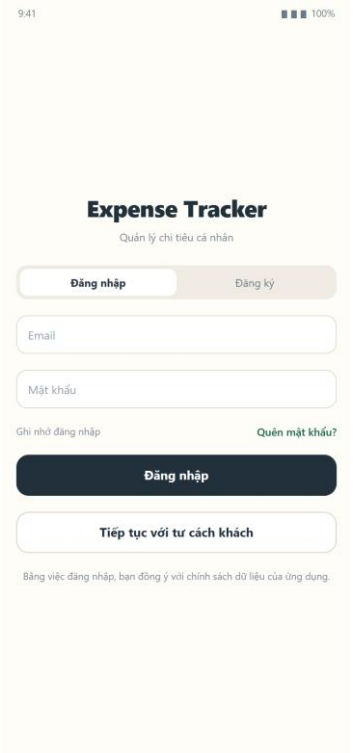


Hình 7. Sơ đồ tuần tự luồng sửa và xoá giao dịch

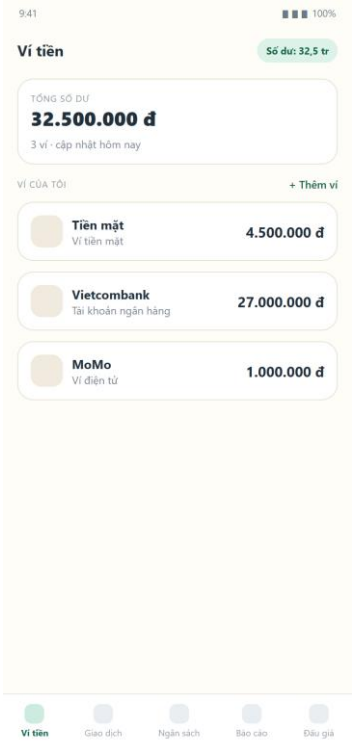
Khi sửa, số dư không được gán lại theo giá trị mới mà phải điều chỉnh theo hiệu số: lấy số tiền có dấu của bản mới trừ số tiền có dấu của bản cũ. Khi xoá thì hoàn lại đúng số tiền đã trừ. Công thức hiệu số nằm trong `transaction_balance.dart` của `lib/features/transaction/data/`, viết dưới dạng hàm thuần để test không cần Firestore.

3.4. Wireframe các màn hình (Câu 4)

Bảy màn hình dưới đây phủ toàn bộ luồng sử dụng. Bản vẽ dùng khung 390 × 844 dp tương đương một điện thoại phổ thông. Wireframe trình bày **bố cục và thứ tự thông tin**; bảng màu chốt khi lập trình là xanh lá (`#2E7D32`) theo design system trong `lib/core/constants/app_colors.dart`, nên màu trên bản vẽ chỉ mang tính ước lệ. Các khối xám là vùng nội dung động, chữ là nhãn thật sẽ hiển thị trên ứng dụng.



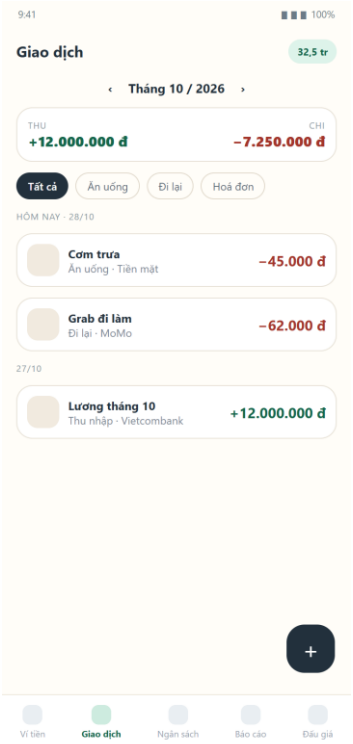
Hình 8. Wireframe màn đăng nhập và đăng ký



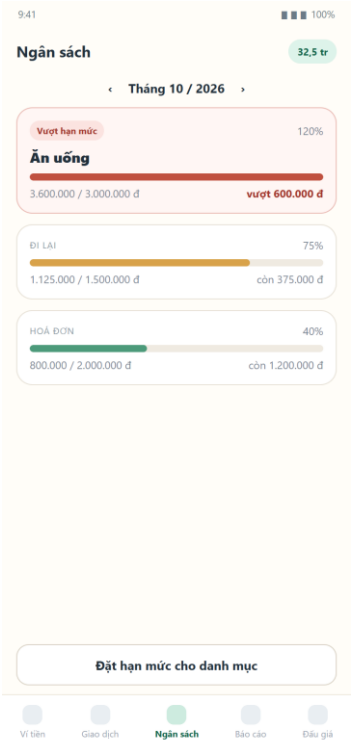
Hình 9. Wireframe màn tổng quan danh sách ví



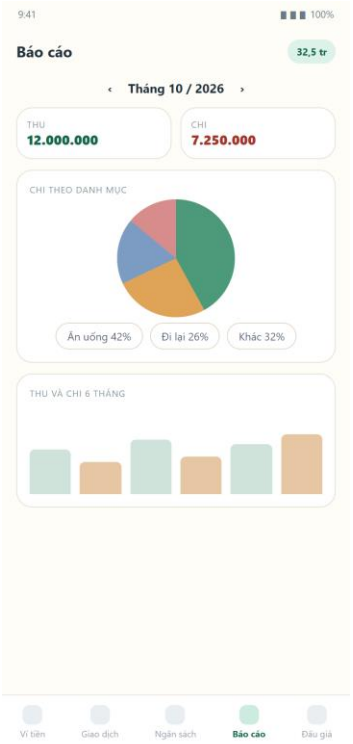
Hình 10. Wireframe màn thêm giao dịch



Hình 11. Wireframe màn danh sách giao dịch



Hình 12. Wireframe màn ngân sách có cảnh báo vượt



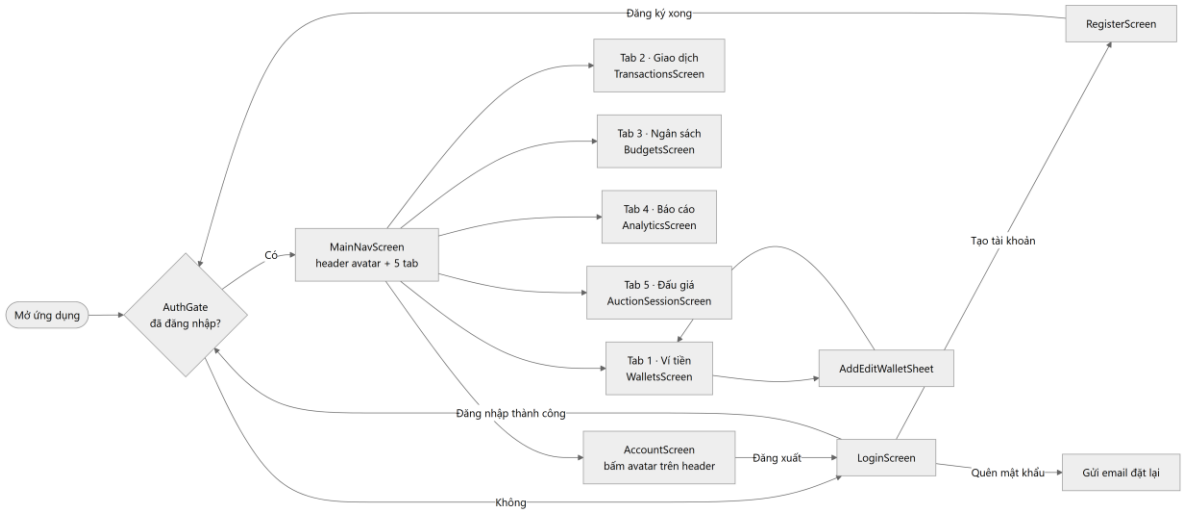
Hình 13. Wireframe màn báo cáo và biểu đồ



Hình 14. Wireframe màn hồ sơ và đăng xuất

3.5. Luồng làm việc của từng màn hình

Sơ đồ dưới đây mô tả đường đi giữa các màn hình. Ba màn xác thực nằm ngoài MainNavScreen vì chỉ hiển thị khi chưa đăng nhập; các màn còn lại truy cập qua năm tab hoặc qua nút trên thanh tiêu đề.



Hình 15. Sơ đồ điều hướng giữa các màn hình

Bảng 10. *Luồng làm việc của từng màn hình*

Màn hình	Lối vào	Chức năng chính	Lối ra
LoginScreen	Mở ứng dụng khi chưa đăng nhập	Đăng nhập, chuyển sang đăng ký, gửi email đặt lại mật khẩu	MainNavScreen
RegisterScreen	Nút "Tạo tài khoản" ở màn đăng nhập	Tạo tài khoản bằng email, mật khẩu, tên hiển thị	MainNavScreen
WalletsScreen	Tab 1	Xem tổng số dư, thêm, sửa, xoá ví	Sheet thêm/sửa ví
TransactionsScreen	Tab 2	Chọn tháng, lọc danh mục, xem tổng thu chi, sửa, xoá	Sheet thêm/sửa giao dịch
BudgetsScreen	Tab 3	Đặt hạn mức cho danh mục, xem tiến độ, nhận cảnh báo	Sheet đặt hạn mức
AnalyticsScreen	Tab 4	Biểu đồ tròn theo danh mục, biểu đồ cột sáu tháng	—
AuctionSessionScreen	Tab 5	Đếm ngược phiên, đặt giá nhanh, xem lịch sử trả giá	Quay lại tab khác
AccountScreen	Bấm avatar trên thanh tiêu đề	Xem hồ sơ, đăng xuất	LoginScreen

Một quy ước nhỏ khi thiết kế luồng: mọi thao tác thêm hoặc sửa đều mở bottom sheet thay vì đẩy sang trang mới. Sheet giữ được ngữ cảnh phía sau, người dùng thấy

danh sách vẫn nằm đó, và đóng sheet là quay lại đúng vị trí cuộn cũ. Chỉ những màn có nội dung dài như phiên đấu giá hay màn tài khoản mới dùng trang riêng.

Chương 4. Cài đặt

4.1. Môi trường và cấu trúc mã nguồn

Bảng 11. *Môi trường phát triển*

Thành phần	Phiên bản / cấu hình
Hệ điều hành phát triển	Windows 10 Pro 22H2 (19045)
Flutter SDK	3.47.4, kênh stable
Dart SDK	3.13.1
Thiết bị chạy thử	Chrome (bản web), emulator Android
Cơ sở dữ liệu	Cloud Firestore, project-mobile-c0faa
Quản lý mã nguồn	Git, mô hình nhánh feature → develop → main
Công cụ	VS Code, Android Studio, Flutter DevTools

Ứng dụng chạy thử trên Chrome và emulator Android. **Không** chạy trên Windows/Linux desktop vì Firebase không cung cấp plugin cho hai nền tảng này; đây là lý do bảng môi trường ở trên chỉ ghi web và Android.

Mã nguồn chia theo feature, mỗi feature tự chứa ba lớp của mình. Cách chia này giúp ba thành viên làm song song mà ít chạm vào tệp của nhau; chỉ những tệp dùng chung như model, collection, theme mới nằm ngoài thư mục feature.

```
lib/
├── core/            hằng số, theme, widget dùng chung, tiện ích
├── data/firestore/  model 5 lớp + collection có withConverter + seeder
├── features/
│   ├── auth/        đăng nhập, đăng ký, AuthGate, màn Tài khoản
│   ├── wallet/      ví tiền
│   ├── transaction/ giao dịch (Member 2)
│   ├── category/    danh mục (Member 2)
│   ├── budget/      ngân sách (Member 3)
│   ├── analytics/   biểu đồ báo cáo (Member 3)
│   ├── auction/     màn hình Phiên Đấu Giá (Câu 5)
│   └── main_nav/    khung ứng dụng và năm tab
└── main.dart        khởi tạo Firebase và ProviderScope
```

Đoạn mã 1. *Cấu trúc thư mục mã nguồn*

Hai thư mục `transaction/` và `category/` hiện chỉ có interface repository, `budget/` và `analytics/` chỉ có màn hình rỗng; phần cài đặt đầy đủ nằm ở nhánh của Member 2 và Member 3. Bản draft giữ đúng cấu trúc thư mục để khi gộp nhánh không phải sửa lại báo cáo.

4.2. Màn hình Phiên Đấu Giá (Câu 5)

Màn hình được dựng lại theo ảnh mẫu: thanh tiêu đề có tên phiên, ảnh sản phẩm lớn ở giữa, thẻ thông tin gồm mã phiên và thời gian bắt đầu – kết thúc, đồng hồ đếm ngược viền đen, bảng lịch sử trả giá và nút "Đặt Giá Nhanh" ghim ở đáy. Màn hình nằm trong tab thứ năm của thanh điều hướng chung. Toàn bộ phần dựng giao diện nằm trong `lib/features/auction/presentation/auction_session_screen.dart`.

Phần khó nhất là đồng hồ đếm ngược. Đồng hồ dùng `Timer.periodic` một giây một lần, trừ dần biến `_remaining`, và phải huỷ trong `dispose` để không rò rỉ bộ nhớ khi người dùng rời màn hình. Khi thời gian về 0 thì dừng timer, màn hình chuyển sang trạng thái đã kết thúc và nút đặt giá bị vô hiệu.

```
void _onTick(Timer timer) {
  if (_remaining.inSeconds <= 0) {
    timer.cancel();
    return;
  }
  setState(() {
    _remaining -= const Duration(seconds: 1);
  });
}
```

Đoạn mã 2. *Đồng hồ đếm ngược của phiên đấu giá*

Nút "Đặt Giá Nhanh" cộng thêm một bước giá cố định (100.000 đồng) vào giá hiện tại và chèn một dòng mới lên đầu lịch sử, kèm nhãn "Bạn" để phân biệt với các lượt trả giá khác. Dữ liệu phiên và lịch sử hiện được giữ trong bộ nhớ, không ghi Firestore, vì đây là màn hình mẫu để luyện giao diện chứ không phải chức năng nghiệp vụ của ứng dụng quản lý chi tiêu. Hướng phát triển là lưu phiên và lượt trả giá thành hai collection riêng, khi đó mới cần khoá lạc quan để tránh hai người cùng thắng một giá.



Hình 16. Màn hình Phiên Đấu Giá chạy trên ứng dụng (Câu 5)

4.3. Triển khai 15 user story (Câu 6)

Bảng dưới đây là bản đồ từ user story sang màn hình và tệp mã nguồn. Cột tệp ghi tệp chứa phần xử lý chính, không liệt kê các tệp phụ như provider hay model. Cột trạng thái khớp với bảng ở mục 2.2.

Bảng 12. *Ánh xạ user story sang màn hình và tệp mã nguồn*

Mã	Chức năng	Màn hình	Tệp xử lý chính	Trạng thái
US01	Đăng ký tài khoản	RegisterScreen	auth/data/auth_service.dart	Đã chạy
US02	Đăng nhập	LoginScreen	auth/data/auth_service.dart	Đã chạy
US03	Quên mật khẩu	LoginScreen	auth/data/auth_service.dart	Đã chạy
US04	Đăng xuất	AccountScreen	auth/data/auth_service.dart	Đã chạy

Mã	Chức năng	Màn hình	Tệp xử lý chính	Trạng thái
US05	Tạo ví	AddEditWalletSheet	wallet/data/repositories/firestore_wallet_repository.dart	Đã chạy
US06	Sửa và xoá ví	WalletsScreen	wallet/presentation/wallets_screen.dart	Đã chạy
US07	Thêm giao dịch	AddEditTransactionSheet	transaction/presentation/add_edit_transaction_sheet.dart	Đã chạy
US08	Sửa giao dịch	AddEditTransactionSheet	transaction/data/repositories/firestore_transaction_repository.dart	Đã chạy
US09	Xoá giao dịch	TransactionsScreen	transaction/presentation/transactions_screen.dart	Đã chạy
US10	Xem theo tháng	TransactionsScreen	transaction/providers/transaction_providers.dart	Đã chạy
US11	Lọc theo danh mục	TransactionsScreen	transaction/providers/transaction_filter.dart	Đã chạy
US12	Tạo danh mục riêng	CategoriesScreen	category/presentation/categories_screen.dart	Đã chạy
US13	Đặt hạn mức tháng	BudgetsScreen	budget/data/repositories/firestore_budget_repository.dart	Đã chạy
US14	Cảnh báo vượt hạn mức	BudgetsScreen	budget/data/budget_progress.dart	Đã chạy
US15	Biểu đồ thu chi	AnalyticsScreen	analytics/presentation/analytics_screen.dart	Đã chạy

4.3.1. Xác thực và tài khoản (US01 – US04)

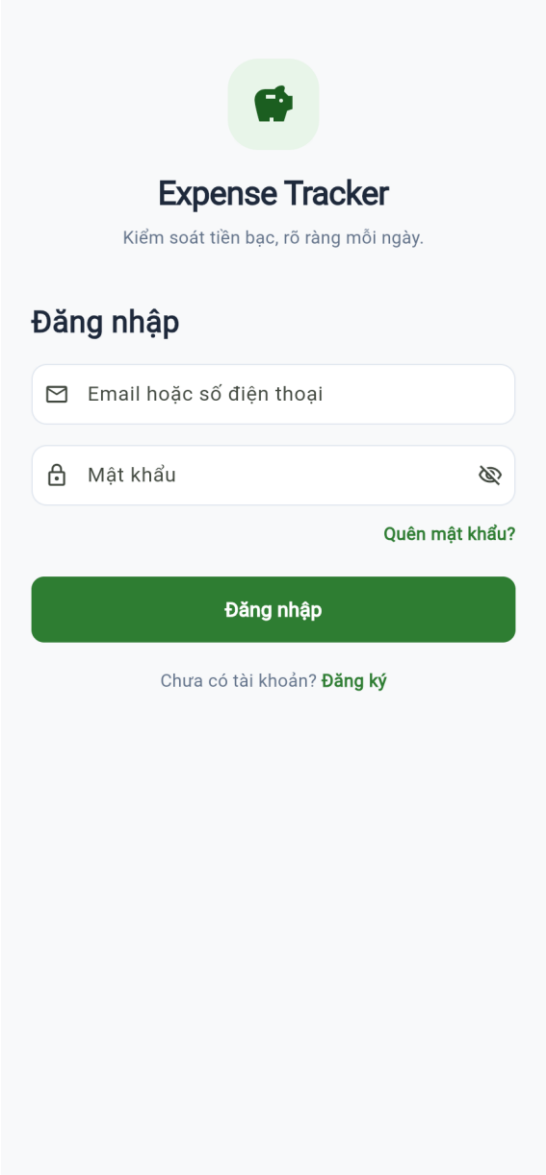
Đăng ký gồm hai bước: tạo tài khoản trên Firebase Authentication rồi ghi hồ sơ vào `users/{uid}`. Nếu bước thứ hai thất bại, tài khoản vẫn tồn tại nhưng thiếu hồ sơ; vì vậy khi đọc trạng thái đăng nhập, `authStateChanges` kiểm tra document có tồn tại hay không và tự dựng hồ sơ mặc định nếu thiếu, tránh trường hợp người dùng đăng nhập được nhưng ứng dụng trắng màn hình.

```
Future<void> register({
  required String email,
  required String password,
  required String displayName,
}) async {
  final credential = await auth.createUserWithEmailAndPassword(
    email: email,
    password: password,
  );
  final user = AppUser(
    id: credential.user!.uid,
    email: email,
    displayName: displayName,
    currencyCode: 'VND',
    createdAt: DateTime.now(),
  );
  await usersCollection(db).doc(user.id).set(user);
}
```

Đoạn mã 3. *Đăng ký tài khoản và ghi hồ sơ*

Đăng nhập chỉ gọi `signInWithEmailAndPassword`; việc điều hướng do AuthGate lo. Quên mật khẩu gọi `sendPasswordResetEmail`, thông báo trung tính "Nếu email tồn tại, hệ thống đã gửi liên kết" để không tiết lộ email nào đã đăng ký —

chi tiết này liên quan tới Chương 6. Nút Quên mật khẩu bị khoá 60 giây sau mỗi lần gửi (`_resetCooldown`) để tránh đốt quota email của Firebase. Đăng xuất gọi `signOut` từ màn Tài khoản, có hộp thoại xác nhận; AuthGate nhận sự kiện và tự quay về màn đăng nhập.



Hình 17. Màn đăng nhập chạy trên Chrome

4.3.2. Quản lý ví (US05 – US06)

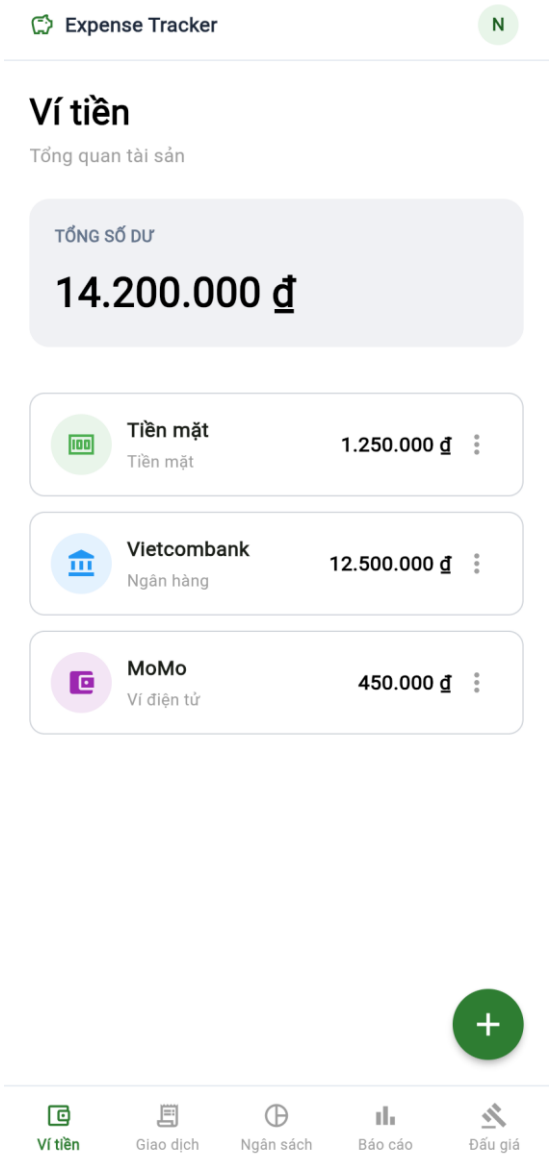
Màn ví hiển thị tổng số dư của tất cả ví trên thẻ đầu tiên và danh sách ví bên dưới. Số dư chỉ đọc, không cho sửa tay: khi thêm ví thì nhập số dư ban đầu, còn sau đó số dư chỉ đổi qua giao dịch. Form thêm ví kiểm tra tên không rỗng và số dư ban đầu là số không âm.

Xoá ví có hộp thoại xác nhận nêu rõ tên ví và cảnh báo hành động không hoàn tác được. Toàn bộ lời gọi Firestore nằm trong `try/catch`, lỗi được hiện bằng SnackBar kèm nội dung lỗi thay vì để màn hình rơi vào trạng thái trắng.

```
Future<void> updateBalance(String walletId, double delta) async {
  // FieldValue.increment cộng/trừ ngay trên server, tránh race condition.
  await walletsCollection(_db)
      .doc(walletId)
      .update({'balance': FieldValue.increment(delta)});
}

@override
```

Đoạn mã 4. *Cập nhật số dư bằng phép cộng nguyên tử trên server*

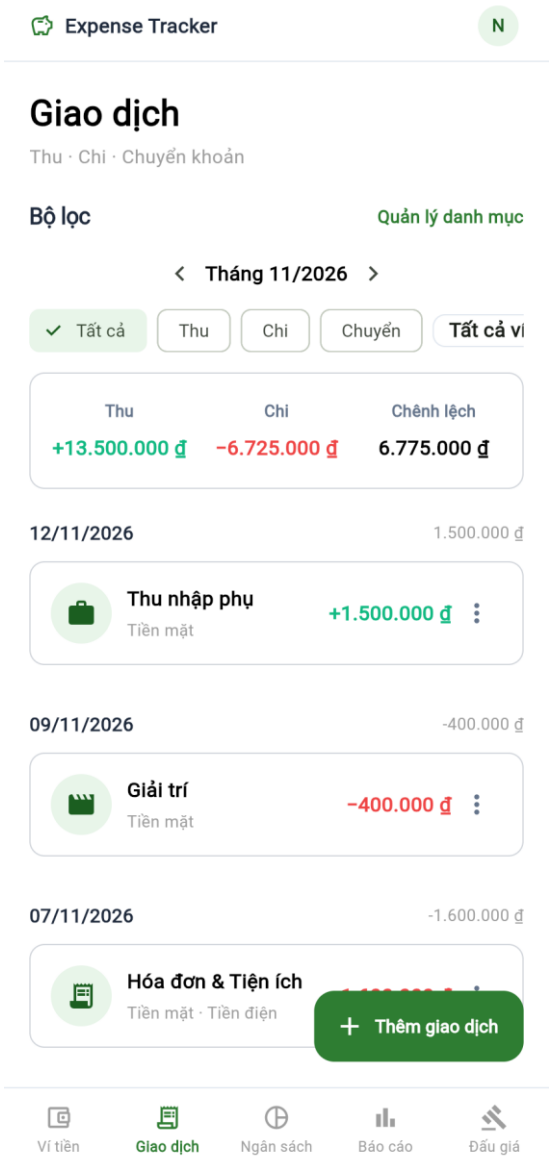


Hình 18. Màn quản lý ví chạy trên ứng dụng

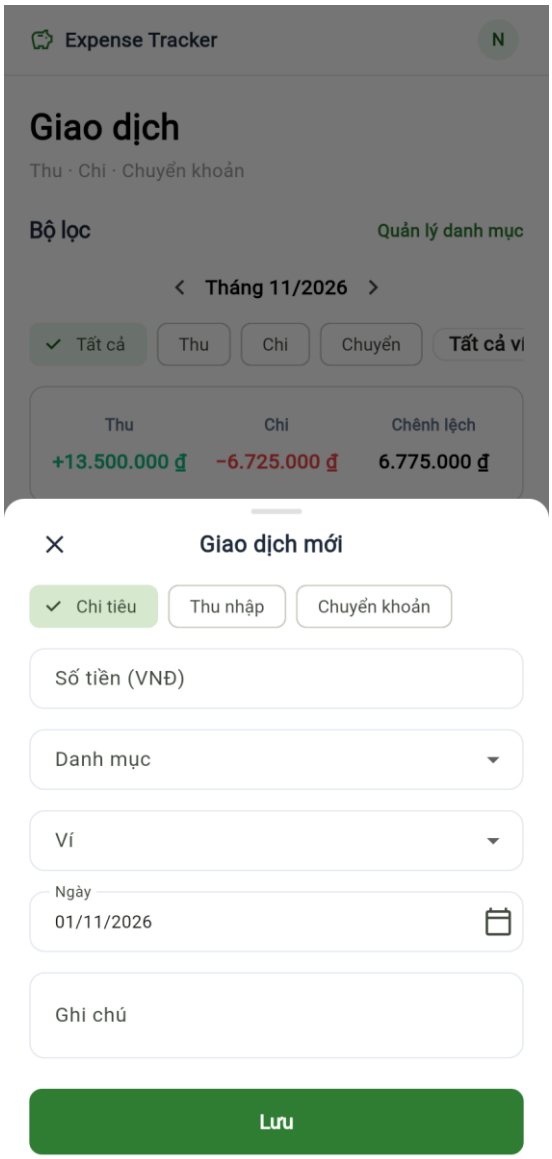
4.3.3. Giao dịch thu chi (US07 – US11)

Màn danh sách có bộ chọn tháng, ba ô tổng thu – chi – chênh lệch, hàng chip lọc theo loại, ví và danh mục, rồi danh sách nhóm theo ngày. Mỗi dòng hiển thị icon danh mục, tên khoản, ví và số tiền có dấu; chạm vào dòng mở sheet sửa, menu ba chấm có Sửa và Xoá. Dữ liệu đọc qua `FirestoreTransactionRepository`; số dư ví cập nhật bằng `FieldValue.increment` sau mỗi thao tác.

Form thêm giao dịch kiểm tra ba điều kiện: số tiền lớn hơn 0, đã chọn ví, và với chuyển khoản thì ví nhận phải khác ví nguồn. Lỗi Firestore hiện bằng SnackBar và form không đóng, nên người dùng không mất dữ liệu vừa nhập.



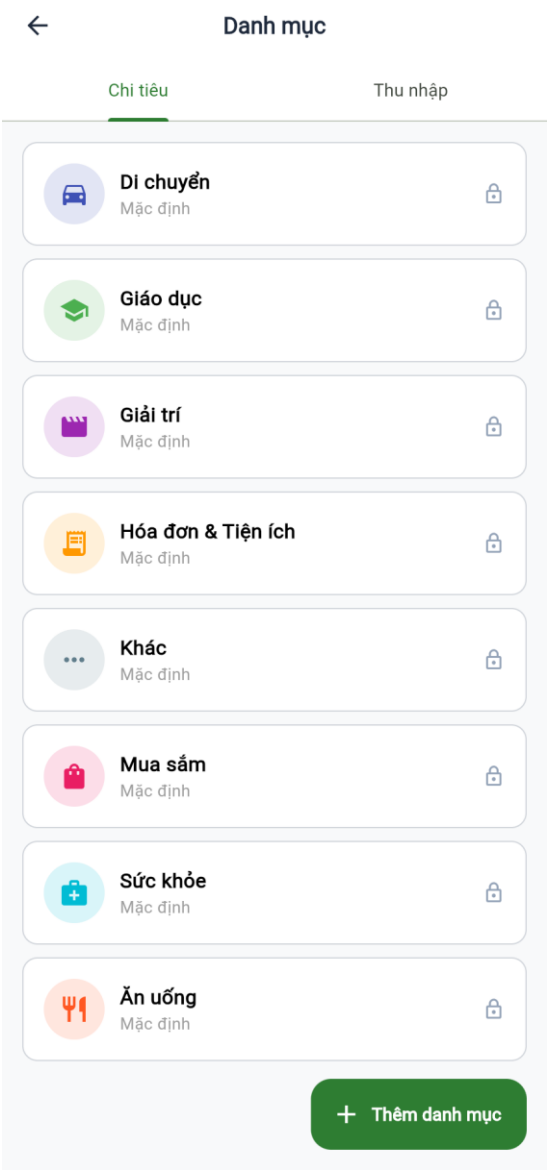
Hình 19. Tab Giao dịch với dữ liệu mẫu: bộ lọc, tổng tháng và danh sách theo ngày



Hình 20. Form thêm giao dịch trong bottom sheet

4.3.4. Danh mục (US12)

Màn danh mục chia hai nhóm: 13 danh mục mặc định do seeder tạo và danh mục người dùng tự tạo. Danh mục mặc định không cho xoá vì giao dịch cũ đang tham chiếu tới; danh mục tự tạo thì sửa được tên, icon và màu. Seeder dùng id cố định nên chạy lại nhiều lần không sinh bản ghi trùng. Màn `CategoriesScreen` mở từ nút "Quản lý danh mục" ở tab Giao dịch, có hai tab Chi tiêu / Thu nhập và FAB thêm danh mục.



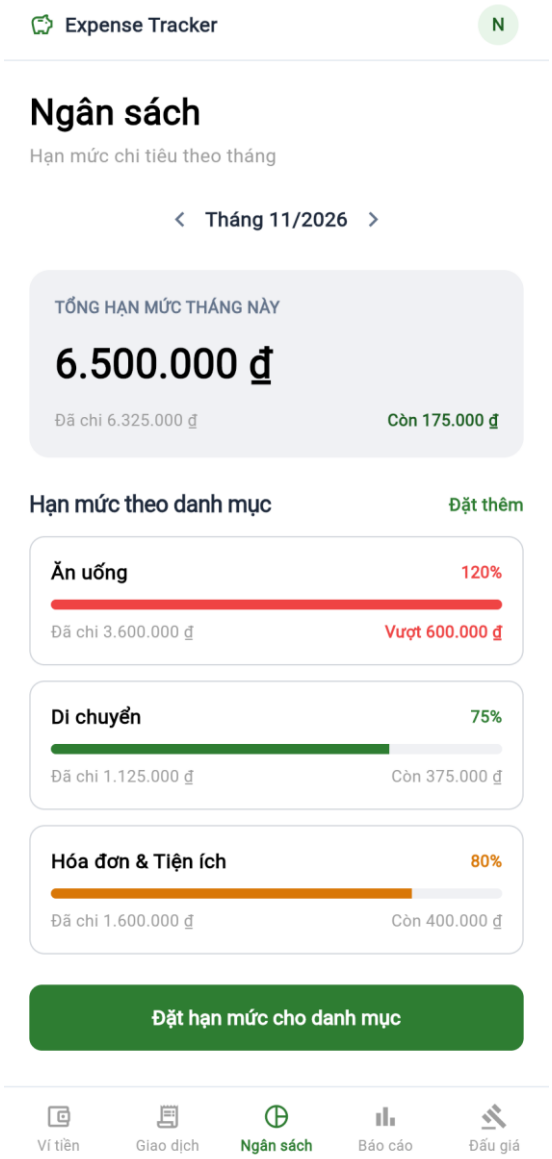
Hình 21. Màn Danh mục: danh mục mặc định có ổ khoá, FAB thêm danh mục

4.3.5. Ngân sách và cảnh báo (US13 – US14)

Ngân sách được tính lại mỗi lần dữ liệu thay đổi, không lưu sẵn số đã chi. Ba mức hiển thị: dưới 80% dùng thanh màu xanh và hiện số tiền còn lại; từ 80% đến 100% dùng màu vàng kèm nhãn "sắp chạm hạn mức"; trên 100% dùng màu đỏ, hiện số vượt và một banner ở đầu danh sách. Ngưỡng 80% chọn theo kinh nghiệm vì còn khoảng một phần năm hạn mức là mốc người dùng còn kịp điều chỉnh chi tiêu.

Mã nguồn gồm `FirestoreBudgetRepository` (5 hàm, lọc `userId` + `month` + `year`), lớp thuần `BudgetProgress` tính tỉ lệ đã chi, provider `budgetProgressProvider` ghép ngân sách với tổng chi theo danh mục, màn `BudgetsScreen` (tab 3) và `BudgetDetailScreen`. Form đặt hạn mức là bottom

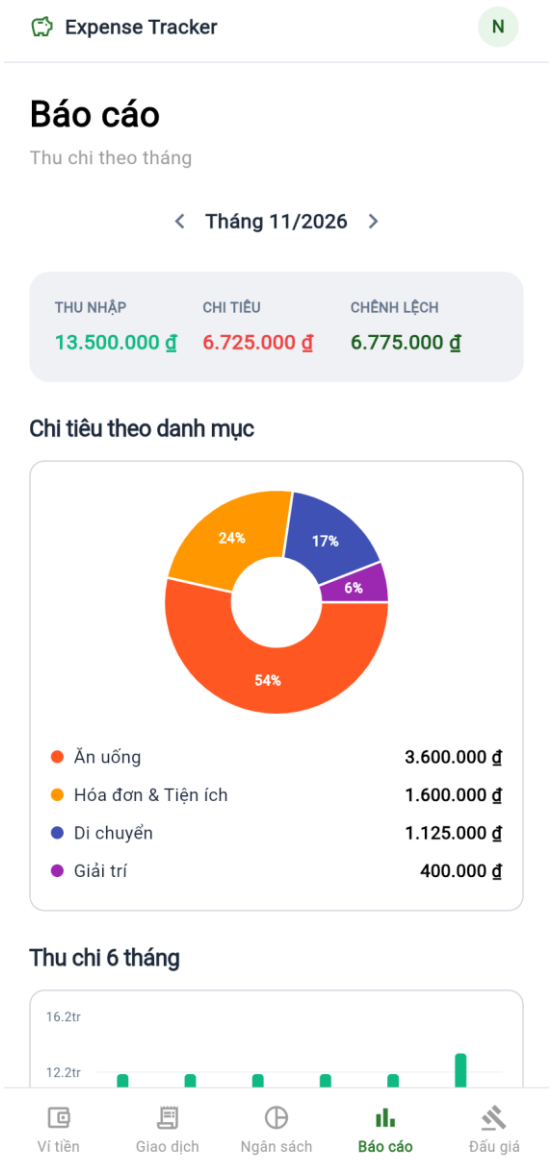
sheet dùng chung cho tạo mới và sửa; xoá có hộp thoại xác nhận, mọi thao tác Firestore đều bọc `try/catch`.



Hình 22. Tab Ngân sách với dữ liệu mẫu: ba mức xanh – vàng – đỏ

4.3.6. Báo cáo và biểu đồ (US15)

Màn báo cáo gồm ba thẻ tổng hợp (thu, chi, chênh lệch), biểu đồ tròn tỉ trọng chi theo danh mục và biểu đồ cột so sánh thu – chi sáu tháng. Dữ liệu lấy từ `transactionsByMonthProvider` và `transactionsByCategoryProvider` của Member 2; màu lấy từ `colorHex` của danh mục nên khớp với icon trong danh sách giao dịch. Provider `sixMonthTotalsProvider` đọc sáu tháng gần nhất để vẽ cột.



Hình 23. Tab Báo cáo với dữ liệu mẫu: biểu đồ tròn và cột

4.4. Firebase và tầng ORM Firestore (Câu 7)

4.4.1. Cấu trúc dữ liệu

Năm collection đặt phẳng ở cấp gốc. Ba collection `wallets`, `transactions`, `budgets` mang trường `userId` và mọi câu đọc đều lọc `where('userId', isEqualTo: uid)`, nên mỗi người chỉ thấy dữ liệu của mình. Cách đặt phẳng giữ được đường dẫn ngắn, cho phép query theo `month`/`year` mà không phải duyệt subcollection, và khớp với Security Rules hiện có.

Bảng 13. *Năm vị trí lưu dữ liệu trên Firestore*

Đường dẫn	Model	Trường chính	Ghi chú
users/{uid}	AppUser	email, displayName, currencyCode,	Mã document = uid

Đường dẫn	Model	Trường chính	Ghi chú
		createdAt	
wallets/{id}	Wallet	userId, name, type, balance, colorHex	Số dư chỉ đổi qua updateBalance
transactions/{id}	Transaction	userId, walletId, categoryId, type, amount, date	Lọc theo userId rồi theo tháng
budgets/{id}	Budget	userId, categoryId, month, year, limitAmount	Khoá nghiệp vụ là bộ ba
categories/{id}	Category	name, type, iconCode, colorHex, isDefault	Dùng chung, không có userId

4.4.2. Ánh xạ document sang model bằng withConverter

Mỗi collection được khai báo một lần kèm `withConverter`, nên khi đọc trả về thẳng đối tượng `Wallet` thay vì `Map`, khi ghi nhận thẳng đối tượng. Nhờ vậy không còn chỗ nào trong giao diện phải tự gọi `data()['balance']` hay tự đổi `Timestamp` sang `DateTime`. Đây là phần ORM mà Câu 7 yêu cầu.

```
CollectionReference<Wallet> walletsCollection(FirebaseFirestore db) {
  return db
      .collection(FirestoreCollections.wallets)
      .withConverter<Wallet>(
        fromFirestore: (snapshot, _) =>
            Wallet.fromDocument(snapshot.id, snapshot.data()!),
        toFirestore: (wallet, _) => wallet.toDocument(),
      );
}
```

Đoạn mã 5. *Collection gắn withConverter cho model Wallet*

Chiều ngược lại nằm trong model: `Wallet.fromDocument(id, data)` đọc từng trường với giá trị mặc định khi thiếu, `toDocument()` đổi `DateTime` sang `Timestamp` và bỏ các trường null. Bộ chuyển đổi hai chiều này được kiểm thử trực tiếp trong `test/wallet_repository_test.dart` (bốn test đầu).

4.4.3. Repository tách khỏi giao diện

Mỗi nhóm dữ liệu có một interface và một bản cài đặt Firestore; riêng ví còn có thêm bản cài đặt trong bộ nhớ (`MockWalletRepository`) dùng cho chế độ mock và widget test. Giao diện chỉ biết interface, nên khi cần chạy thử không có mạng hoặc viết widget test, chỉ cần override provider bằng bản cài đặt trong bộ nhớ. Đây cũng là lý do ba tệp test hiện có chạy được mà không cần khởi tạo Firebase.

4.4.4. Cập nhật số dư bằng FieldValue.increment

Thêm một giao dịch cần hai thao tác: tạo document giao dịch và cộng hoặc trừ số dư ví. Lệnh thứ hai không đọc số dư rồi ghi lại giá trị mới, mà gửi `FieldValue.increment(delta)` để Firestore cộng trên server. Nhờ vậy hai thao tác liên tiếp không ghi đè lên nhau. Với thao tác sửa, số dư được điều chỉnh theo hiệu số giữa bản mới và bản cũ; với thao tác xoá thì hoàn lại đúng số tiền đã trừ. Hạn chế còn lại là hai lệnh chưa nằm trong cùng một batch — xem mục 3.3.2 và hướng khắc phục ở 7.3.

4.4.5. Security Rules

Rules được viết theo nguyên tắc mặc định từ chối: không có luật nào cho phép thì mọi truy cập đều bị chặn. Ba collection dữ liệu riêng yêu cầu `resource.data.userId == request.auth.uid` khi đọc và `request.resource.data.userId == request.auth.uid` khi tạo mới, nên không thể ghi hộ người khác. Collection `categories` dùng chung nên chỉ yêu cầu đã đăng nhập.

```
rules_version = '2';

// Firestore Security Rules — Expense Tracker (project-mobile-c0faa)
// Nguyên tắc: mỗi người dùng chỉ đọc/ghi được Document của chính mình.
// - wallets / transactions / budgets: Document phải có userId == uid đang đăng nhập.
// - categories: collection dùng chung (13 danh mục mặc định), user đã đăng nhập được đọc/ghi.
// - users: mỗi uid chỉ truy cập hồ sơ của chính mình.

service cloud.firestore {
  match /databases/{database}/documents {

    match /users/{userId} {
      allow read, write: if request.auth != null && request.auth.uid == userId;
    }

    match /wallets/{walletId} {
      allow read: if request.auth != null && resource.data.userId == request.auth.uid;
      allow create: if request.auth != null && request.resource.data.userId ==
request.auth.uid;
      allow update, delete: if request.auth != null && resource.data.userId ==
request.auth.uid;
    }

    match /transactions/{transactionId} {
      allow read: if request.auth != null && resource.data.userId == request.auth.uid;
      allow create: if request.auth != null && request.resource.data.userId ==
request.auth.uid;
      allow update, delete: if request.auth != null && resource.data.userId ==
request.auth.uid;
    }

    match /budgets/{budgetId} {
      allow read: if request.auth != null && resource.data.userId == request.auth.uid;
      allow create: if request.auth != null && request.resource.data.userId ==
```

```
request.auth.uid;
      allow update, delete: if request.auth != null && resource.data.userId ==
request.auth.uid;
    }

    match /categories/{categoryId} {
      allow read: if request.auth != null;
      allow write: if request.auth != null;
    }
  }
}
```

Đoạn mã 6. *Firestore Security Rules của dự án*

Một chi tiết đáng chú ý: truy vấn không kèm điều kiện `userId` sẽ bị từ chối chứ không trả về danh sách rỗng, vì Firestore kiểm tra rule trên từng document ứng viên. Vì thế mọi repository đều buộc phải lọc theo uid — quy tắc này được ghi trong `docs/Huong-Dan-Firestore-Cho-Member.md` và đã áp dụng trong `FirestoreWalletRepository`.

4.5. Kiểm tra dữ liệu và xử lý lỗi (Câu 8a)

Có ba tầng chặn lỗi. Tầng thứ nhất là validator trên form, chặn dữ liệu sai ngay khi người dùng bấm lưu. Tầng thứ hai là try/catch quanh mọi lời gọi Firestore, đổi mã lỗi kỹ thuật thành câu tiếng Việt. Tầng thứ ba là Security Rules, chặn cả trường hợp giao diện bị sửa hoặc gọi trực tiếp API.

Bảng 14. *Các tình huống lỗi và thông báo tương ứng*

Tình huống	Cách phát hiện	Thông báo cho người dùng
Bỏ trống email	Validator trên form	Vui lòng nhập email
Email sai định dạng	Validator (có @ và dấu chấm)	Email không hợp lệ
Mật khẩu dưới 6 ký tự	Validator độ dài	Mật khẩu tối thiểu 6 ký tự
Nhập lại mật khẩu không khớp	So sánh hai ô	Mật khẩu nhập lại không khớp
Tên ví để trống	Validator trên sheet	Vui lòng nhập tên ví
Số dư ban đầu âm hoặc không phải số	Parse + kiểm tra	Số dư không hợp lệ
Sai email hoặc mật khẩu	Mã lỗi invalid-credential / wrong-password / user-not-found	Email hoặc mật khẩu không đúng.
Gửi lại email quá nhiều lần	Mã lỗi too-many-requests	Thử quá nhiều lần. Vui lòng thử lại sau.
Mất mạng khi ghi	catch tổng quát	Lỗi khi xóa ví: … (kèm nội dung lỗi)
Không có quyền đọc dữ liệu	Mã lỗi permission-denied	Phiên đăng nhập hết hạn, đăng nhập lại

```
  String _authErrorMessage(fa.FirebaseAuthException e) {
    switch (e.code) {
```

```
      case 'invalid-credential':
      case 'wrong-password':
      case 'user-not-found':
        return 'Email hoặc mật khẩu không đúng.';
      case 'invalid-email':
        return 'Email không hợp lệ.';
      case 'too-many-requests':
        return 'Thử quá nhiều lần. Vui lòng thử lại sau.';
      default:
        return e.message ?? 'Đăng nhập thất bại.';
    }
  }
```

Đoạn mã 7. *Đổi mã lỗi Firebase thành câu tiếng Việt*

Ngoài thông báo lỗi, ứng dụng còn phân biệt ba trạng thái giao diện: đang tải thì hiện vòng xoay, có dữ liệu thì hiện nội dung, lỗi thì hiện thông báo kèm nút thử lại. Cách xử lý này tránh màn hình trắng khi mạng chậm, vốn là lỗi hay gặp ở ứng dụng đọc dữ liệu từ xa.

Chương 5. Kiểm thử

5.1. Chiến lược kiểm thử

Nhóm kiểm thử theo bốn mức, mỗi mức trả lời một câu hỏi khác nhau. Unit test hỏi "hàm này tính đúng không". Widget test hỏi "màn hình này hiển thị và phản ứng đúng không". Kiểm thử thủ công hỏi "người dùng thật có gặp lỗi nào không". Kiểm thử giao diện hỏi "màn hình có đúng bố cục và trạng thái rỗng đúng không". Bốn mức này tương ứng với các ý 8a – 8e của đề bài.

Bảng 15. *Bốn mức kiểm thử*

Mức	Công cụ	Phạm vi	Số test
Unit test (8b)	flutter_test, không cần Firebase	Model, ORM hai chiều, CRUD và số dư ví trong bộ nhớ	78
Widget test (8c)	flutter_test + provider override	Toàn bộ shell ứng dụng ở chế độ mock: 5 tab, danh sách ví	27
Kiểm thử giao diện (8e)	Chrome + Playwright, ảnh chụp từng màn	Bố cục, trạng thái rỗng, màu sắc, vị trí nút	7 màn
Thủ công (8a, 8d)	Chạy tay trên Chrome và emulator	Các trường hợp lỗi, dữ liệu biên, mất mạng	15 kịch bản

Điểm quan trọng là phần lớn test chạy được mà **không cần khởi tạo Firebase**. Repository được khai báo qua interface, nên trong test chỉ cần override provider bằng bản cài đặt trong bộ nhớ. Nhóm chọn tự viết lớp giả thay vì dùng thư viện `fake_cloud_firestore` vì phiên bản Firebase 7.0.0 chưa tương thích ổn định với thư viện này, và bản giả tự viết kiểm soát được cả trường hợp lỗi mạng.

5.2. Unit test (8b)

Unit test hiện có nằm ở bảy tệp, tập trung vào bốn nhóm logic dễ sai nhất: chuyển đổi dữ liệu hai chiều giữa model và document, các phép tính trên số dư ví, tiến độ ngân sách, và logic lọc giao dịch.

Bảng 16. *Các nhóm unit test hiện có*

Tệp test	Nội dung kiểm tra	Số test
test/generics_test.dart	Lớp generic `Genericclass<T>` và `GenericEntityManager`: getter/setter, kế thừa, quản lý đa hình	4
test/wallet_repository_test.dart	ORM `fromDocument`/`toDocument` của Wallet, dựng lại document thiếu trường; CRUD và `updateBalance` trên `MockWalletRepository`	7
test/budget_repository_test.dart	ORM Budget hai chiều, `copyWith`; tiến độ ngân sách ở mức thường, 80%, vượt hạn mức và hạn mức 0	8

Tệp test	Nội dung kiểm tra	Số test
test/category_repository_test.dart	ORM Category, CRUD trên mock repository, kiểm tra trùng tên trong cùng loại	13
test/transaction_repository_test.dart	ORM Transaction, CRUD và điều chỉnh số dư ví khi thêm/sửa/xoá, giao dịch chuyển khoản	30
test/category_providers_test.dart	Provider danh mục: lọc theo loại, cập nhật sau khi thêm/xoá	4
test/transaction_providers_test.dart	Bộ lọc giao dịch theo tháng, loại, ví, danh mục và tổng thu chi	12

Ba trường hợp biên được kiểm tra riêng. Thứ nhất là document thiếu trường: `fromDocument` phải trả về giá trị mặc định thay vì ném lỗi, vì dữ liệu cũ có thể chưa có trường mới thêm. Thứ hai là `updateBalance` với id không tồn tại: hàm phải im lặng bỏ qua thay vì làm sập luồng giao diện. Thứ ba là `deleteWallet` chỉ được xoá đúng ví được chọn, không đụng tới ví khác trong danh sách.

Bảy tệp trên phủ toàn bộ tầng dữ liệu và provider; ba tệp còn lại là widget test cho màn hình, được trình bày ở mục 5.3.

5.3. Widget test (8c)

Ngoài bài smoke test dựng cả ứng dụng bên dưới, hai tệp `category_screen_test.dart` (11 test) và `transaction_screen_test.dart` (15 test) kiểm thử màn Danh mục và Giao dịch: trạng thái rỗng/loading/lỗi, bộ lọc theo tháng và loại, validate form, thêm/sửa/xoá kèm hộp thoại xác nhận.

Bài smoke test dựng toàn bộ ứng dụng ở chế độ mock, override `walletRepositoryProvider` bằng `MockWalletRepository.seeded()` và `currentUserProvider` bằng một người dùng giả, rồi kiểm tra những gì người dùng nhìn thấy ở màn đầu tiên.

Bảng 17. *Kịch bản widget test*

Kịch bản	Kỳ vọng	Kết quả
Dựng MyApp với dữ liệu mock	Không ném exception, không có widget nào bị tràn khung	Đạt
Kiểm tra tiêu đề màn ví	Có chữ "Ví tiền" và "Tổng quan tài sản"	Đạt
Kiểm tra thẻ tổng số dư	Có nhãn "TỔNG SỐ DƯ"	Đạt
Kiểm tra danh sách ví mẫu	Có "Tiền mặt", "Vietcombank", "MoMo"	Đạt
Kiểm tra thanh điều hướng	Có đủ 5 nhãn: Ví tiền, Giao dịch, Ngân sách, Báo cáo, Đấu giá	Đạt

Một lưu ý khi viết widget test cho ứng dụng có đồng hồ đấu giá: phải dùng `pump(Duration(seconds: 1))` thay vì chờ thời gian thật, và phải `pump` thêm một lần sau khi widget bị huỷ để kiểm tra timer đã được cancel. Nếu không, test sẽ báo lỗi "A Timer is still pending" và chỉ ra đúng chỗ rò rỉ.

5.4. Phân tích ca lỗi và ca thành công (8d)

Bảng dưới đây liệt kê các trường hợp đã chạy và quan sát được, tách riêng ca thành công và ca lỗi. Cột "Xử lý trong mã" chỉ ra chỗ nào trong mã nguồn chịu trách nhiệm cho hành vi đó.

Bảng 18. *Ca thành công và ca lỗi đã quan sát*

Ca	Dữ liệu vào	Kết quả quan sát	Xử lý trong mã
Thành công	Đăng ký email mới, mật khẩu 6 ký tự	Tạo tài khoản, vào thẳng màn chính	`register()` + AuthGate
Thành công	Đăng nhập đúng	Vào màn chính, dữ liệu riêng của uid	`signIn()` + `where(userId)`
Thành công	Tạo ví 1.000.000 đồng	Ví mới xuất hiện, tổng số dư tăng	`insertWallet()`
Thành công	Đổi tên ví	Tên mới hiện, số dư giữ nguyên	`updateWallet()`
Thành công	Xoá ví sau khi xác nhận	Ví biến mất, tổng số dư giảm	`deleteWallet()`
Thành công	Bấm Đặt Giá Nhanh trong phiên	Giá tăng 100.000, thêm dòng "Bạn"	`_placeBid()`
Lỗi	Đăng nhập sai mật khẩu	SnackBar "Email hoặc mật khẩu không đúng."	`invalid-credential`
Lỗi	Email sai định dạng	Lỗi ngay dưới ô nhập, không gọi Firebase	Validator form
Lỗi	Mật khẩu 4 ký tự	Lỗi "Mật khẩu tối thiểu 6 ký tự"	Validator form
Lỗi	Nhập lại mật khẩu khác	Lỗi "Mật khẩu nhập lại không khớp"	Validator form
Lỗi	Tên ví để trống	Lỗi "Vui lòng nhập tên ví", sheet không đóng	Validator sheet
Lỗi	Số dư ban đầu là chữ	Lỗi "Số dư không hợp lệ"	`double.tryParse`
Lỗi	Bấm Quên mật khẩu khi ô email trống	SnackBar "Nhập email trước…"	`_forgotPassword()`
Lỗi	Bấm Quên mật khẩu hai lần liên tiếp	Nút bị khoá 60 giây	`_resetCooldown`
Lỗi	Mất mạng khi xoá ví	SnackBar "Lỗi khi xóa ví: …"	`try/catch` trong `_confirmDelete`
Lỗi	Mở tab Giao dịch / Ngân sách / Báo cáo	Hiện trạng thái rỗng, không crash	`EmptyState`

Nhận xét từ bảng trên: các ca lỗi phía người dùng đều bị chặn **trước** khi gọi Firebase, nên không tốn lượt đọc/ghi và không để lại dữ liệu rác. Các ca lỗi phía mạng

được bắt ở tầng repository và hiện bằng SnackBar, không làm mất dữ liệu đã nhập trên form.

5.5. Bảng test case thủ công (8a)

Bảng dưới đây ghi các kịch bản chạy tay trên ứng dụng, gồm cả trường hợp thành công và trường hợp lỗi. Cột Thực tế ghi kết quả quan sát được tại lần chạy ngày 01/11/2026 trên nhánh `develop` sau khi gộp đủ ba thành viên.

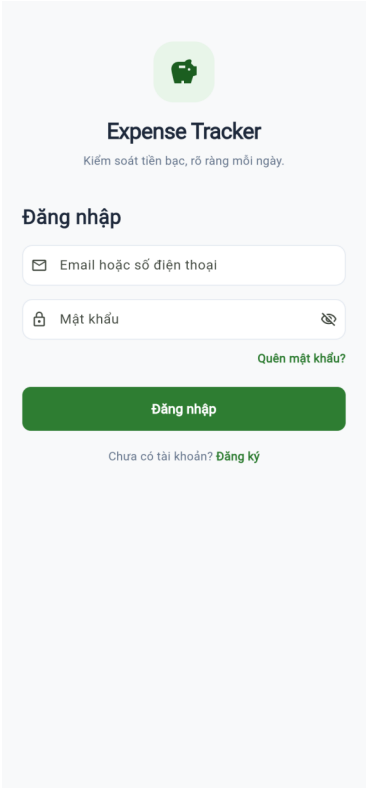
Bảng 19. *Bảng test case thủ công*

STT	Kịch bản	Dữ liệu vào	Kỳ vọng	Thực tế
1	Đăng ký tài khoản mới	email hợp lệ, mật khẩu 6 ký tự	Vào thẳng màn chính	Đạt
2	Đăng ký với email đã dùng	email đã tồn tại	Báo lỗi của Firebase	Đạt
3	Đăng nhập sai mật khẩu	mật khẩu sai	Báo email hoặc mật khẩu không đúng	Đạt
4	Quên mật khẩu	email đã đăng ký	Gửi mail, thông báo trung tính, khoá nút 60s	Đạt
5	Tạo ví mới	tên, loại, số dư 1.000.000	Ví xuất hiện, tổng số dư tăng	Đạt
6	Sửa tên ví	đổi tên	Tên mới hiển thị, số dư giữ nguyên	Đạt
7	Xoá ví	bấm Xoá, xác nhận	Ví biến mất, tổng số dư giảm	Đạt
8	Đăng xuất	bấm Đăng xuất, xác nhận	Quay về màn đăng nhập	Đạt
9	Mở Phiên Đấu Giá	tab 5	Đồng hồ chạy, đặt giá tăng 100.000	Đạt
10	Đóng ứng dụng khi đang đếm ngược	chuyển tab khác	Không lỗi, timer được huỷ	Đạt
11	Thêm giao dịch chi	50.000, Ăn uống, Tiền mặt	Dòng mới, số dư giảm 50.000	Đạt
12	Sửa giao dịch chi thành thu	đổi loại, giữ số tiền	Số dư tăng thêm 100.000	Đạt
13	Lọc giao dịch theo danh mục	chọn Ăn uống	Chỉ còn giao dịch Ăn uống	Đạt
14	Đặt hạn mức 3.000.000 cho Ăn uống	đã chi 3.600.000	Banner vượt hạn mức, thanh đỏ	Đạt
15	Xem báo cáo tháng có dữ liệu	chọn tháng 10	Hai biểu đồ hiện, tổng đúng	Đạt

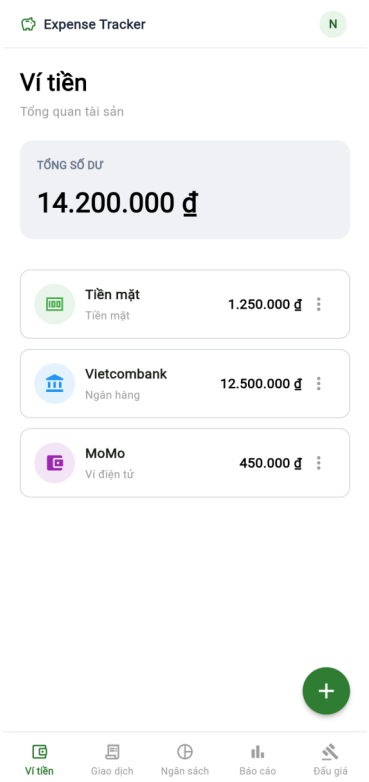
5.6. Kiểm thử giao diện màn hình (8e)

Phần kiểm thử giao diện được thực hiện bằng cách build bản web, phục vụ ở cổng nội bộ rồi chụp từng màn ở kích thước 430 × 932 (tỉ lệ một điện thoại phổ thông). Ảnh chụp dùng dữ liệu mẫu, không dùng dữ liệu thật của thành viên. Bảy ảnh dưới đây là các màn chính; ảnh đầy đủ được lưu trong

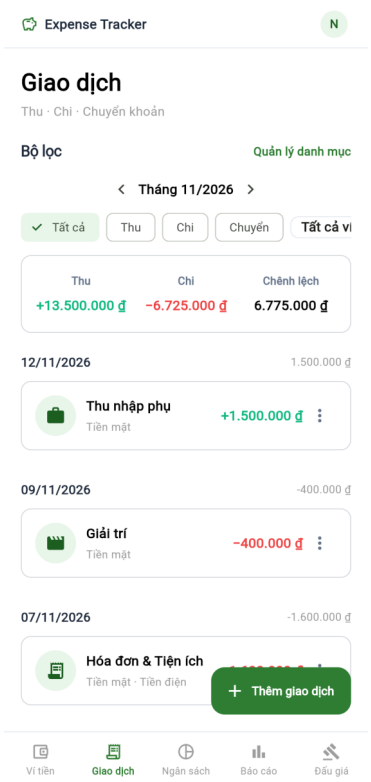
`docs/docs/mobile/assets/app/` và sinh lại bằng `docs/docs/mobile/tools/capture_app.py`.



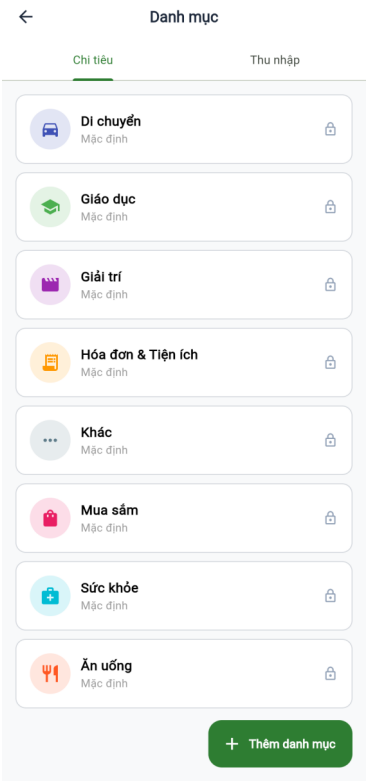
Hình 24. Màn đăng nhập



Hình 25. Tab Ví tiền



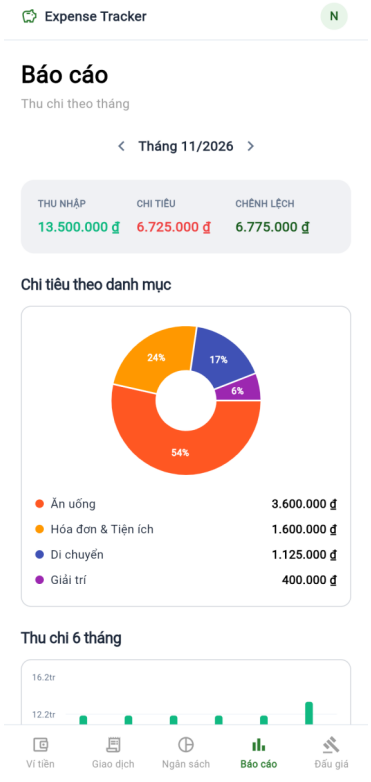
Hình 26. Tab Giao dịch



Hình 27. Màn Danh mục



Hình 28. Tab Ngân sách



Hình 29. Tab Báo cáo



Hình 30. Tab Đấu giá

Các điểm đã kiểm tra trên ảnh: tiêu đề lớn 28 đậm và dòng mô tả xám đúng quy ước; số tiền chỉ đặt trên nền trắng hoặc xám rất nhạt; màu số tiền theo đúng quy ước (thu xanh, chi đỏ); thanh điều hướng có đủ năm tab và tab đang chọn tô xanh `#2E7D32`; không có chữ nào bị cắt hoặc tràn khung ở chiều rộng 430 px.

Cả bốn tab dữ liệu đều đã có nội dung; trạng thái rỗng vẫn được giữ cho tài khoản mới, có icon, tiêu đề và dòng mô tả nên người dùng không nhầm là ứng dụng hỏng.

5.7. Tổng hợp kết quả

Lần chạy gần nhất (`flutter test`, ngày 01/11/2026) có **105 test** và toàn bộ đạt: 78 unit test và 27 widget test. Thời gian chạy khoảng 9 giây, không test nào phải khởi tạo Firebase nên chạy được cả trên máy không có mạng.

Hạn chế của phần kiểm thử là chưa có test tự động cho Security Rules, chưa có `integration_test` chạy trên thiết bị thật và chưa đo độ phủ dòng lệnh. Ba phần này được để ở hướng phát triển (mục 7.3) vì cần cấu hình Firebase Emulator và công cụ đo phủ riêng. Toàn bộ 15 user story đều có test hoặc kịch bản kiểm thử tương ứng trong bảng 5.2 và 5.5.

Chương 6. Đạo đức và quyền riêng tư dữ liệu

6.1. Vì sao ứng dụng tài chính phải đặt riêng tư lên trước

Dữ liệu tài chính cá nhân khác dữ liệu thông thường ở ba điểm. Thứ nhất, nó tiết lộ gần như toàn bộ đời sống: ăn ở đâu, đi lại thế nào, khám bệnh khi nào, nuôi con hay chăm người già. Thứ hai, nó không thể thay đổi như mật khẩu — một khi lộ, người dùng không thể "đổi lại" lịch sử chi tiêu của mình. Thứ ba, nó có giá trị thương mại trực tiếp nên luôn có động cơ thu thập thêm. Vì vậy với ứng dụng quản lý chi tiêu, nguyên tắc tối thiểu dữ liệu không chỉ là yêu cầu kỹ thuật mà là điều kiện để người dùng dám nhập số tiền thật.

Trong phạm vi bài tập, nhóm tiếp cận vấn đề này ở mức thiết kế và mã nguồn: dữ liệu nào cần lưu, ai đọc được, lưu bao lâu, xoá bằng cách nào. Ba câu hỏi đó được trả lời cụ thể ở mục 6.4 sau khi điểm lại sự kiện đang được thảo luận trên lớp và hai khung pháp lý liên quan.

6.2. Sự việc 22/10/2026: Facebook bị kiện 25 tỷ USD về dữ liệu cá nhân

Nội dung thảo luận đầu giờ ngày 22/10/2026 nêu việc Facebook bị kiện với số tiền yêu cầu bồi thường lên tới **25 tỷ USD** vì vi phạm quyền riêng tư dữ liệu (data privacy). Con số này lớn hơn nhiều lần các mức xử phạt hành chính từng được áp dụng trước đây, và đó chính là điều đáng chú ý: rủi ro pháp lý của dữ liệu cá nhân đã chuyển từ "chi phí tuân thủ" sang "rủi ro có thể làm sụp một mảng kinh doanh".

Sự việc này không đứng riêng lẻ. Năm 2018, vụ Cambridge Analytica cho thấy dữ liệu của khoảng 87 triệu người dùng bị thu qua một bài trắc nghiệm tâm lý, trong đó người làm bài đồng ý chia sẻ dữ liệu và kéo theo dữ liệu của bạn bè họ — những người bị ảnh hưởng không hề biết và không hề đồng ý. Năm 2019, Ủy ban Thương mại Liên bang Mỹ áp mức phạt 5 tỷ USD cho cùng chuỗi sự việc. Điểm chung của cả hai lần là: dữ liệu được thu **hợp pháp về hình thức** nhưng **không có ý nghĩa về đạo đức**, vì người dùng không hiểu mình đang cho phép cái gì, cho ai và trong bao lâu.

Với nhóm làm ứng dụng, bài học rút ra có ba tầng. Tầng kỹ thuật: dữ liệu chỉ được thu khi thật sự cần cho chức năng đang có. Tầng tổ chức: phải biết dữ liệu nằm ở đâu, ai có quyền đọc, ai có quyền xoá. Tầng đạo đức: nếu một tính năng chỉ có lợi cho nhóm phát triển mà không mang lại giá trị cho người dùng, thì không làm — kể cả khi điều khoản dịch vụ cho phép.

Nhóm cũng thẳng thắn nhìn vào giới hạn của mình: một bài tập lớn không thể đánh giá được toàn bộ rủi ro pháp lý của một sản phẩm thật. Nhưng những quyết định nhỏ trong phạm vi bài tập — không thêm SDK quảng cáo, không xin quyền vị trí, không hiển thị "email này chưa đăng ký" — là cùng một loại quyết định mà một sản phẩm thật phải làm ở quy mô lớn hơn.

6.3. Khung pháp lý

6.3.1. GDPR của Liên minh châu Âu

Quy định chung về bảo vệ dữ liệu (GDPR, EU 2016/679) áp dụng từ ngày 25 tháng 5 năm 2018 và là khuôn mẫu cho nhiều quốc gia sau này. Ba điểm đáng chú ý với một ứng dụng di động: dữ liệu phải được thu cho mục đích xác định và tối thiểu; người dùng có quyền truy cập, sửa, xoá và chuyển dữ liệu của mình; mức phạt có thể lên tới 20 triệu euro hoặc 4% doanh thu toàn cầu, lấy mức cao hơn.

6.3.2. Pháp luật Việt Nam

Nghị định 13/2023/NĐ-CP về bảo vệ dữ liệu cá nhân, ban hành ngày 17 tháng 4 năm 2023 và có hiệu lực từ ngày 1 tháng 7 năm 2023, là văn bản toàn diện đầu tiên về lĩnh vực này. Nghị định chia dữ liệu thành dữ liệu cá nhân cơ bản và dữ liệu cá nhân nhạy cảm; dữ liệu tài chính thuộc nhóm nhạy cảm nên cần sự đồng ý rõ ràng. Chủ thể dữ liệu có các quyền được biết, đồng ý, rút lại sự đồng ý, truy cập, xoá, hạn chế xử lý, phản đối, khiếu nại và yêu cầu bồi thường thiệt hại.

Sau nghị định, Việt Nam đã luật hoá lĩnh vực này bằng Luật Bảo vệ dữ liệu cá nhân, có hiệu lực từ ngày 1 tháng 1 năm 2026. Với ứng dụng trong bài tập, điểm quan trọng là các quyền này phải có đường thực hiện trong sản phẩm chứ không chỉ nằm trên giấy: muốn xoá dữ liệu thì phải có nút xoá dữ liệu thật.

6.4. Áp dụng vào ứng dụng quản lý tài chính cá nhân

6.4.1. Tối thiểu dữ liệu

Ứng dụng chỉ lưu năm loại dữ liệu: hồ sơ (email, tên hiển thị), ví, giao dịch, danh mục, ngân sách. Không lưu số điện thoại, ngày sinh, địa chỉ, vị trí, danh bạ hay ảnh. Mỗi trường đều có chỗ dùng cụ thể trong một màn hình hoặc một phép tính; trường nào không dùng thì không khai báo. Mật khẩu không bao giờ được lưu trong Firestore; Firebase Authentication giữ phần đó dưới dạng băm và ứng dụng không đọc được.

6.4.2. Kiểm soát truy cập bằng Security Rules

Rules ở mục 4.4.5 là lớp bảo vệ thật sự, vì giao diện có thể bị sửa hoặc bị gọi vượt qua. Ba collection `wallets`, `transactions`, `budgets` yêu cầu trường `userId` trùng uid đang đăng nhập ở cả hai chiều đọc và ghi, nên người dùng A không thể đọc hay ghi hộ dữ liệu của B. Collection `categories` dùng chung nên chỉ yêu cầu đã đăng nhập — đây là điểm còn hở: hiện chưa tách danh mục riêng của từng người, nên về lý thuyết một người có thể sửa danh mục mà người khác đang dùng. Hướng khắc phục là thêm trường `ownerId` và cho phép ghi khi `ownerId == uid` hoặc danh mục là mặc định.

6.4.3. Quyền xoá dữ liệu — khoảng trống đã biết

Ứng dụng hiện có đăng xuất nhưng **chưa có chức năng xoá tài khoản và toàn bộ dữ liệu**. Đây là khoảng trống so với quyền xoá dữ liệu mà Nghị định 13/2023 yêu cầu, và nhóm ghi rõ thay vì tuyên bố đã đáp ứng. Thiết kế dự kiến: xoá lần lượt `transactions`, `budgets`, `wallets`, hồ sơ `users/{uid}`, rồi mới xoá tài khoản trên Firebase Authentication. Thứ tự này quan trọng — nếu xoá tài khoản trước, người dùng không còn quyền ghi để xoá dữ liệu còn lại và dữ liệu sẽ nằm lại vĩnh viễn.

6.4.4. Những chỗ cố ý không làm

Nhóm cân nhắc và quyết định không thêm ba tính năng thường gặp. Không gửi email quảng cáo hay nhắc nộp tiền bằng dịch vụ bên thứ ba vì như vậy phải chia sẻ email cho bên khác. Không chặn tính năng xem báo cáo để ép người dùng nâng cấp gói — mô hình "dồn người dùng vào góc" bị coi là dark pattern trong thiết kế sản phẩm. Không hiển thị thông báo kiểu "email này chưa đăng ký" ở màn quên mật khẩu, vì câu đó giúp kẻ tấn công dò email nào đã có tài khoản.

Ba quyết định trên đều làm sản phẩm kém "hiệu quả thương mại" hơn. Nhóm ghi lại vì đó chính là phần đạo đức: chọn điều đúng khi có lựa chọn dễ hơn nhưng gây hại cho người dùng.

6.5. Liên hệ với nội dung thảo luận trên lớp

Các buổi thảo luận đầu giờ tập trung vào ba câu hỏi, và cả ba đều xuất hiện khi nhóm làm dự án này. Câu hỏi thứ nhất là ai sở hữu dữ liệu người dùng tạo ra: nhóm hiểu là người dùng sở hữu, còn nhóm chỉ giữ hộ và phải trả lại hoặc xoá khi được yêu cầu. Câu hỏi thứ hai là sự đồng ý có thật sự tự nguyện không, khi mà để dùng ứng dụng thì buộc phải chấp nhận điều khoản. Cách nhóm xử lý là chỉ xin quyền tối thiểu

cần cho chức năng đang dùng, không gộp mọi quyền vào một lần bấm. Câu hỏi thứ ba là trách nhiệm khi dữ liệu bị lộ: nhóm ghi rõ trong báo cáo những gì đã bảo vệ và những gì chưa, thay vì tuyên bố "an toàn tuyệt đối".

Từ sự việc ngày 22/10/2026, nhóm bổ sung một câu hỏi thứ tư: nếu ứng dụng lớn lên và có nhà đầu tư, nhóm có giữ được nguyên tắc tối thiểu dữ liệu không? Đây là câu hỏi chưa có câu trả lời chắc chắn, nhưng cách trả lời đúng là ghi nguyên tắc vào tài liệu và vào Security Rules ngay từ đầu, để sau này muốn phá cũng phải sửa ở chỗ công khai chứ không phải một dòng mã lén thêm vào.

Cũng từ thảo luận, nhóm thống nhất một quy ước nhỏ khi viết mã: không đặt dữ liệu cá nhân vào tên biến trong log, không in email hay số tiền ra console khi chạy thử, và không đưa dữ liệu thật của thành viên vào ảnh chụp màn hình dùng trong báo cáo. Ảnh chụp trong Chương 4 và 5 dùng dữ liệu mẫu vì lý do đó.

Chương 7. Kết luận và hướng phát triển

7.1. Kết quả đạt được

Tại thời điểm viết báo cáo, cả 15 user story chạy được trên `develop` với Firebase thật: đăng ký, đăng nhập, quên mật khẩu, đăng xuất, quản lý ví, ghi giao dịch, quản lý danh mục, đặt hạn mức và xem báo cáo. Ba kết quả đáng chú ý nhất:

- **Kiến trúc ba lớp có thể kiểm thử**: repository nằm sau interface nên toàn bộ test chạy được mà không cần khởi tạo Firebase, kể cả widget test dựng cả ứng dụng.
- **Số dư ví cập nhật nguyên tử**: mọi thay đổi số dư đi qua `FieldValue.increment`, nên hai lần ghi liên tiếp không ghi đè lên nhau.
- **Dữ liệu tách theo người dùng ở mức cấu trúc**: mọi document trong `wallets`, `transactions`, `budgets` đều mang `userId`, và Security Rules kiểm tra trường này ở cả chiều đọc và ghi.

Về mặt quy trình, nhóm hoàn thành các bước phân tích, thiết kế, cài đặt phần lõi, kiểm thử và viết báo cáo. Bảng ánh xạ ở mục 4.3 cho phép đối chiếu từng user story với màn hình và tệp mã nguồn, thuận tiện khi bảo vệ.

7.2. Hạn chế

- Chưa có đồng bộ ngoại tuyến có chủ đích: Firestore có bộ nhớ đệm nhưng ứng dụng chưa xử lý trường hợp thêm giao dịch khi mất mạng rồi đóng ứng dụng.
- Hai lệnh ghi khi thêm giao dịch (tạo document và cộng số dư) chưa nằm trong cùng một `WriteBatch`.
- Chưa có chức năng xoá tài khoản và dữ liệu, dù Nghị định 13/2023 yêu cầu quyền này.
- Chưa hỗ trợ nhiều tiền tệ trong cùng một tài khoản; toàn bộ số tiền mặc định là VND.
- Chưa có kiểm thử tự động cho Security Rules, chưa có `integration_test` và chưa đo độ phủ dòng lệnh.
- Màn Phiên Đấu Giá là màn hình mẫu, dữ liệu nằm trong bộ nhớ và mất khi tải lại.

7.3. Hướng phát triển

1. Đưa hai lệnh ghi giao dịch vào cùng một `WriteBatch` để loại bỏ trường hợp lệch số dư khi mất mạng giữa chừng.
2. Thêm chức năng xoá tài khoản và dữ liệu theo đúng thứ tự: dữ liệu trước, tài khoản sau.
3. Thêm chế độ ngoại tuyến thật: bật `persistence` của Firestore, xếp hàng thao tác khi mất mạng và hiển thị trạng thái "chờ đồng bộ" trên từng dòng giao dịch.
4. Thêm giao dịch định kỳ (tiền nhà, học phí) để ứng dụng tự sinh giao dịch theo lịch thay vì nhập tay.
5. Thêm xuất dữ liệu ra CSV và sao lưu, phục vụ quyền truy cập dữ liệu của người dùng.
6. Thêm kiểm thử Security Rules bằng Firebase Emulator và đo độ phủ bằng `flutter test --coverage`.

Chương 8. Phân công công việc và đóng góp (Câu 10)

Nhóm ba thành viên chia theo nhóm chức năng, mỗi người sở hữu một nhánh Git và một thư mục mã nguồn để tránh xung đột. Bảng dưới đây ghi phần việc theo tuần; bảng tiếp theo là phần việc chính của từng người.

Bảng 20. *Kế hoạch và phân công theo tuần*

Tuần	Công việc chung	Minh (22010009)	Hải Đăng (24100454)	Thịnh (24100005)
1–2 (8–21/9)	Khảo sát đề tài, chọn công nghệ, chốt mô hình dữ liệu	Dựng khung dự án, cấu hình Firebase, viết 5 model	Đặc tả luồng giao dịch	Đặc tả luồng ngân sách
3–4 (22/9–5/10)	Chốt kiến trúc ba lớp, chuyển từ Drift sang Firestore	AuthService, WalletRepository, design system, seeder	Dựng repository giao dịch (nhánh riêng)	Dựng repository ngân sách (nhánh riêng)
5–6 (6–19/10)	Ghép khung ứng dụng 5 tab	AuthGate, MainNav, màn ví, sửa luồng quên mật khẩu	Màn danh sách + form giao dịch, lọc theo tháng	Màn ngân sách, biểu đồ (nhánh riêng)
7 (20–26/10)	Màn hình mẫu và rà soát	Phiên Đấu Giá (Câu 5), màn Tài khoản, đăng xuất	Rà soát lỗi giao dịch, viết test	Rà soát cảnh báo ngân sách
8 (27/10–2/11)	Báo cáo, video, kiểm thử cuối	Dựng báo cáo, chụp ảnh, quay video, kiểm thử	Viết phần kiểm thử, rà báo cáo	Viết phần thiết kế, rà báo cáo

Bảng dưới đây ghi phần việc chính và sản phẩm bàn giao của từng người. Tỉ lệ đóng góp chính xác được nhóm chốt lại bằng văn bản riêng trước khi nộp bản cứng; số liệu commit thật ở bảng sau dùng để đối chiếu.

Bảng 21. *Đóng góp của từng thành viên*

Thành viên	MSSV	Phần việc chính và sản phẩm bàn giao
Dương Công Minh	22010009	Xác thực, Quản lý ví, Tầng ORM Firestore, Màn hình Phiên Đấu Giá, kiểm thử và báo cáo
Phạm Hải Đăng	24100454	Giao dịch thu/chi, danh mục, lọc và tìm kiếm giao dịch
Đinh Đức Thịnh	24100005	Ngân sách, cảnh báo vượt hạn mức, biểu đồ báo cáo

Bảng dưới đây lấy trực tiếp từ `git log --all` của kho mã nguồn tại thời điểm dựng báo cáo, không nhập tay, và đã loại commit của bot CI. Số liệu tính trên **mọi nhánh**, nên phần việc còn nằm ở nhánh thành viên vẫn được tính.

Bảng 22. *Thống kê commit thật (70 commit, mọi nhánh)*

Tác giả commit	Số commit	Tỉ lệ
Minhne2194	55	79%
Pham Hai Dang	7	10%
Đinh Đức Thịnh	4	6%

Tác giả commit	Số commit	Tỉ lệ
2dawng-206	2	3%
Manx-bit	1	1%
PhamHaiDang06	1	1%

Bảng 23. *Tám commit gần nhất trên develop*

Ngày	Tác giả	Nội dung commit
2026-11-01	Minhne2194	merge: đồng bộ main theo develop (bản phát hành)
2026-11-01	Minhne2194	merge: gộp giao dịch và danh mục của Thành viên 2
2026-10-29	Pham Hai Dang	fix(transaction): test widget không treo, thay pumpAndSettle bằng pump
2026-10-29	Pham Hai Dang	feat(transaction): màn hình danh sách, form thêm/sửa và xóa giao dịch
2026-10-29	Pham Hai Dang	feat(transaction): providers giao dịch và bộ lọc theo tháng/loại/ví/da
2026-10-29	Pham Hai Dang	feat(transaction): repository Firestore và mock, cập nhật số dư ví
2026-10-29	Pham Hai Dang	feat(category): màn hình quản lý danh mục thu/chi
2026-10-29	Pham Hai Dang	feat(category): providers danh mục chạy trên Firestore

Số commit không phản ánh đúng khối lượng vì có giai đoạn cả nhóm làm trên cùng một máy để ghép nhánh, và một số commit gộp nhiều tệp. Vì vậy bảng phân công theo tuần ở trên là căn cứ chính, bảng commit dùng để đối chứng.

Tài liệu tham khảo

[1] Google, "Flutter documentation," 2026. [Online]. Available: https://docs.flutter.dev/. [Accessed: Oct. 20, 2026].

[2] Google, "Cloud Firestore documentation — Data model, transactions and batched writes," 2026. [Online]. Available: https://firebase.google.com/docs/firestore. [Accessed: Oct. 20, 2026].

[3] Google, "Firebase Authentication documentation," 2026. [Online]. Available: https://firebase.google.com/docs/auth. [Accessed: Oct. 20, 2026].

[4] Google, "Firestore Security Rules — Get started," 2026. [Online]. Available: https://firebase.google.com/docs/firestore/security/get-started. [Accessed: Oct. 22, 2026].

[5] Remi Rousselet, "Riverpod documentation," 2026. [Online]. Available: https://riverpod.dev/docs/introduction/getting_started. [Accessed: Oct. 20, 2026].

[6] AppTakk, "fl_chart — A highly customizable Flutter chart library," pub.dev, 2026. [Online]. Available: https://pub.dev/packages/fl_chart. [Accessed: Oct. 22, 2026].

[7] Chính phủ Việt Nam, "Nghị định 13/2023/NĐ-CP về bảo vệ dữ liệu cá nhân," ban hành 17/4/2023, hiệu lực 01/7/2023.

[8] Quốc hội Việt Nam, "Luật Bảo vệ dữ liệu cá nhân," có hiệu lực từ 01/01/2026.

[9] European Parliament and Council, "Regulation (EU) 2016/679 (GDPR)," Official Journal of the European Union, 2016, áp dụng từ 25/5/2018.

[10] C. Cadwalladr and E. Graham-Harrison, "Revealed: 50 million Facebook profiles harvested for Cambridge Analytica in major data breach," The Guardian, 17/3/2018.

[11] Federal Trade Commission, "FTC Imposes $5 Billion Penalty and Sweeping New Privacy Restrictions on Facebook," 24/7/2019.

[12] Nội dung thảo luận đầu giờ ngày 22/10/2026 về vụ kiện Facebook 25 tỷ USD do vi phạm quyền riêng tư dữ liệu (tài liệu học phần CSE702027).

Phụ lục A. Hướng dẫn chạy chương trình

Các bước chạy trên máy đã cài Flutter:

1. Mở terminal tại thư mục `Project-Mobile`.
2. Chạy `flutter pub get` để tải thư viện.
3. Chạy `flutter run -d chrome` để mở trên trình duyệt, hoặc chọn emulator Android trong VS Code.
4. Chạy `flutter test` để thực hiện toàn bộ kiểm thử tự động.

Ứng dụng dùng Firebase project `project-mobile-c0faa`; cấu hình đã có trong `lib/firebase_options.dart`. Muốn chạy giao diện mà không cần mạng, đổi cờ `kUseMockData` trong `lib/main.dart` thành `true` để dùng ví mẫu trong bộ nhớ thay cho Firestore. **Không** chạy trên Windows/Linux desktop vì Firebase không hỗ trợ hai nền tảng này.

```
# Chạy trên trình duyệt (chế độ dữ liệu thật)
flutter run -d chrome


# Chạy toàn bộ kiểm thử
flutter test


# Xuất bản web tĩnh
flutter build web --release
```

Đoạn mã 8. *Các lệnh chạy thường dùng*

Phụ lục B. Khai báo sử dụng công cụ hỗ trợ

Bảng 24. *Công cụ hỗ trợ đã dùng*

Công cụ	Phạm vi sử dụng
Kimi Code (AI coding assistant)	Gợi ý cấu trúc báo cáo, soạn bản nháp văn bản, gợi ý đoạn mã Dart và viết unit test

Công cụ trên được dùng ở mức gợi ý: dựng khung báo cáo, diễn đạt lại nội dung, gợi ý đoạn mã và test. Toàn bộ nội dung đưa vào báo cáo đã được nhóm đọc lại, đối chiếu với mã nguồn và chạy thử trên máy. Các số liệu về kiểm thử, số dòng mã, lịch sử commit trong báo cáo đều lấy từ lần chạy thật của dự án, không lấy từ công cụ AI.